

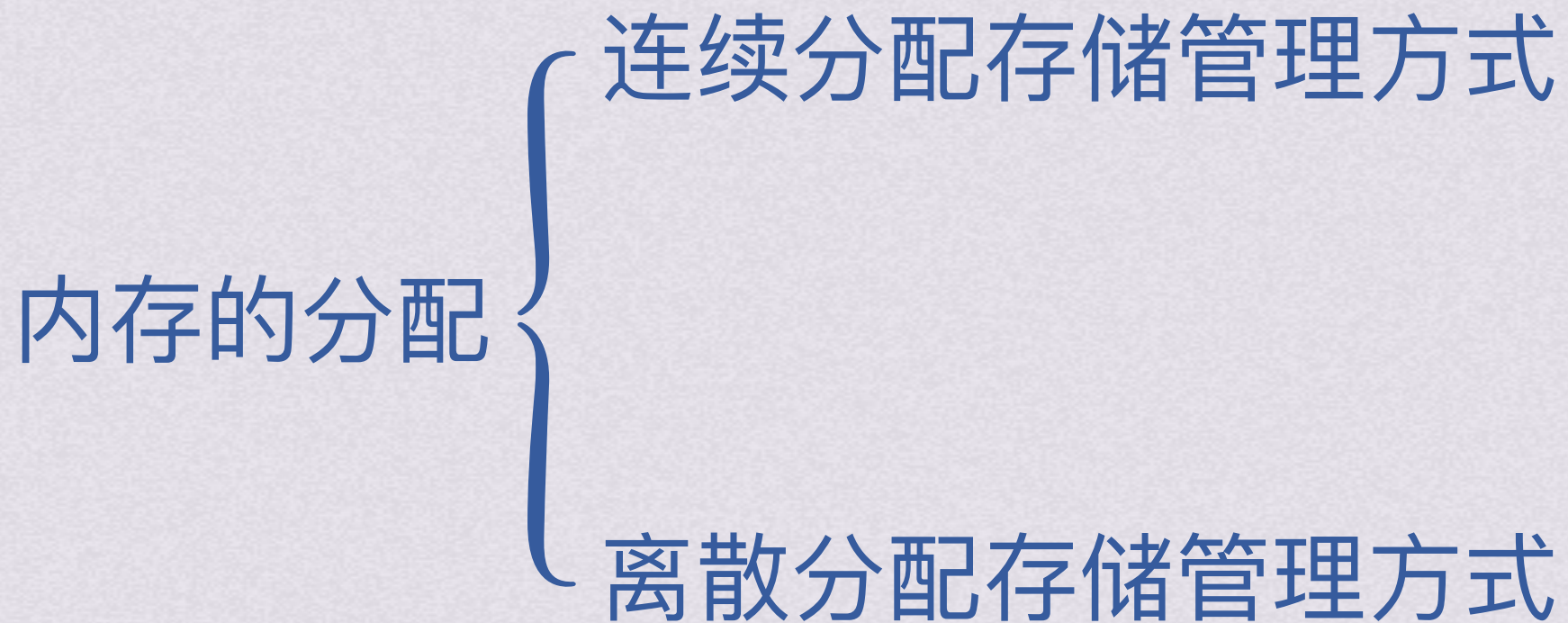
bok25

**基
礎**

課、操作系統



内存的分配





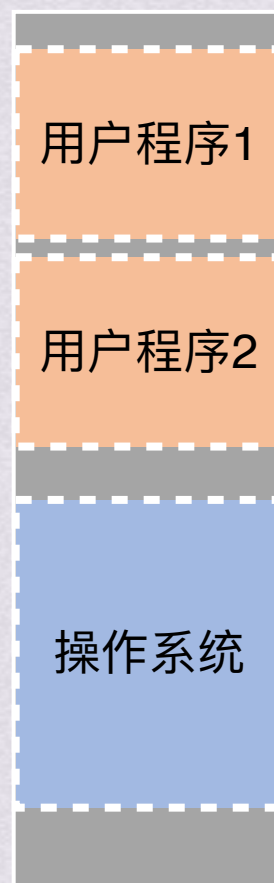
连续分配存储管理方式



连续分配存储管理方式

要把用户程序装入内存，必须给他分配一定内存空间

连续分配方式就是给用户程序分配一个连续的内存空间，程序中代码或数据的逻辑地址相邻，体现在内存空间分配时物理地址的相邻



内存

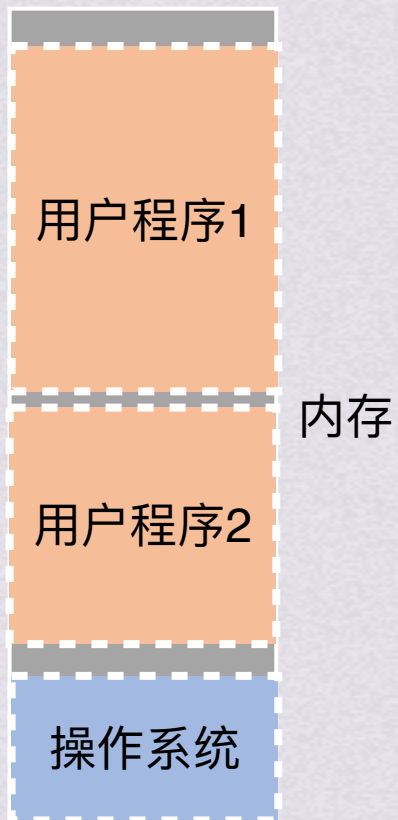


连续分配存储管理方式

内存保护

要把用户程序装入内存，必须给他分配一定内存空间

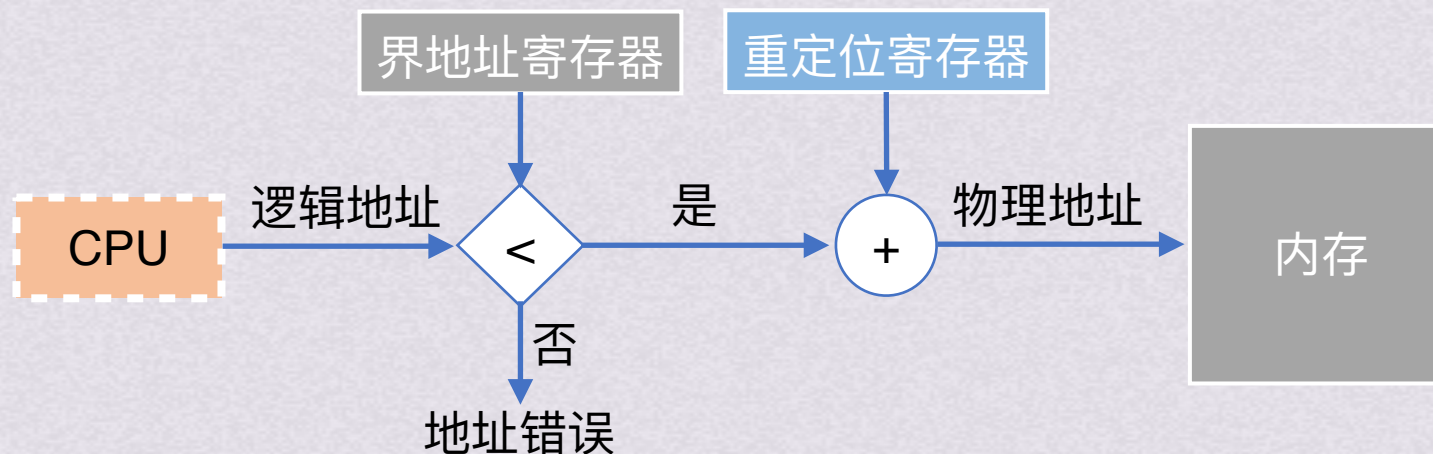
连续分配方式就是给用户程序分配一个连续的内存空间，程序中代码或数据的逻辑地址相邻，体现在内存空间分配时物理地址的相邻



为了保护用户程序之间不会互相受影响，或者操作系统不受其他用户进程的影响，可以用两种方式实现内存保护

1. CPU中设置一对上下限寄存器，分别记录用户进程在主存中的上下限地址，CPU每次要访存的时候都和寄存器对比检查一下有没有越界
2. 采用**重定位寄存器**和**界地址寄存器**（界限寄存器）进行越界检查。**重定位寄存器**中存放的是最小的合法物理内存地址（基地址），**界地址寄存器**中存放的是合法范围的大小（界限地址）

CPU硬件对用户态下产生的物理地址与寄存器的地址进行比较，
即判断“**基地址** $\leq$ **物理地址** $<$ （**基地址**+**界限地址**）”是否成立





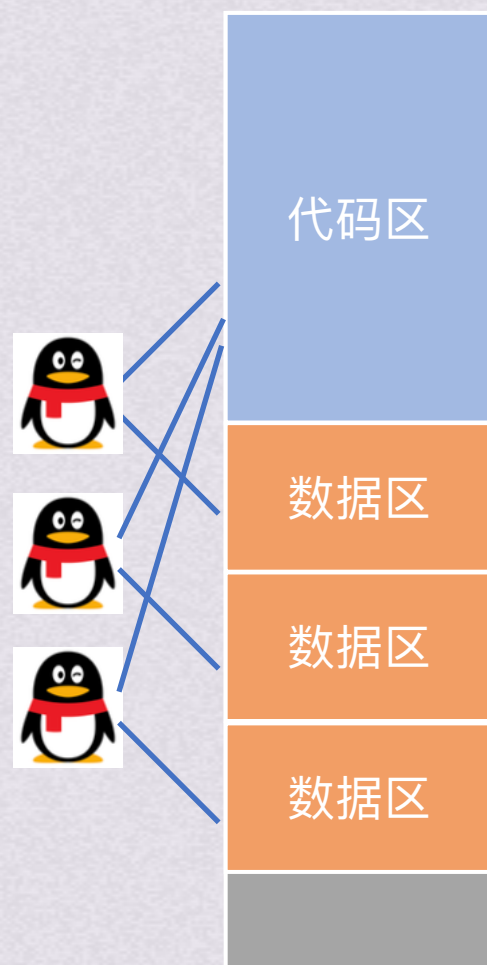
连续分配存储管理方式

内存共享

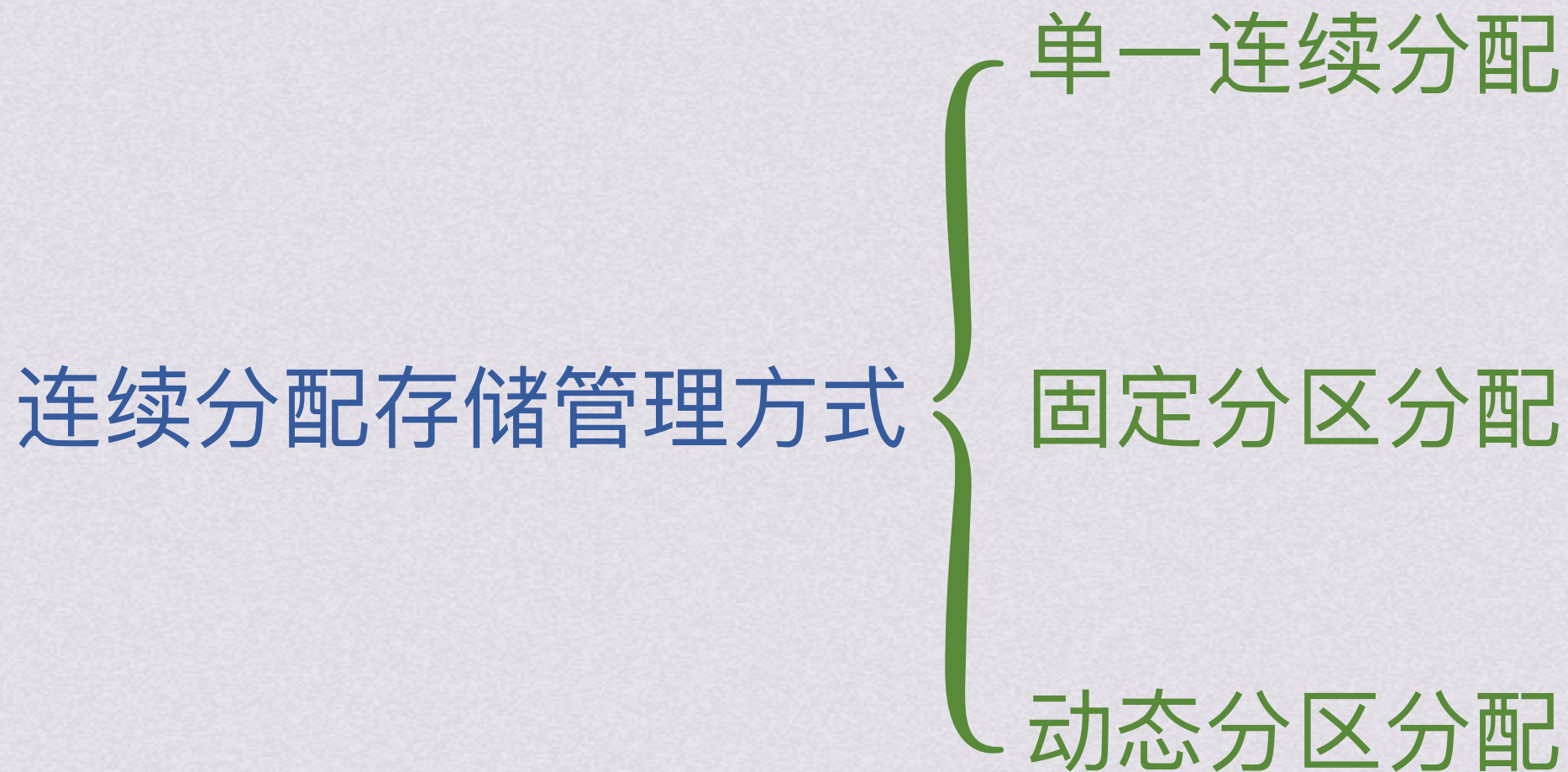
不是所有的进程内存空间都适合共享，只有那些只读的区域才能共享

允许多个进程同时访问但不允许被任何进程修改的代码称为**可重入代码（纯代码）**

这种代码适合共享



这样就可以达到减少多个用户消耗内存的效果





连续分配存储管理方式

单一连续分配

单一连续分配指在单道程序环境下，存储器管理方式是把内存分为系统区和用户区，系统区供OS使用，一般放在内存的低地址部分。而在用户区内存中，仅装有一道用户程序，整个内存的用户空间由该程序独占



{ 固定分区分配
动态分区分配

{ 离散分配存储管理方式



连续分配存储管理方式

单一连续分配

{ 固定分区分配
动态分区分配

{ 离散分配存储管理方式



连续分配存储管理方式

固定分区分配

{ 单一连续分配
动态分区分配

{ 离散分配存储管理方式

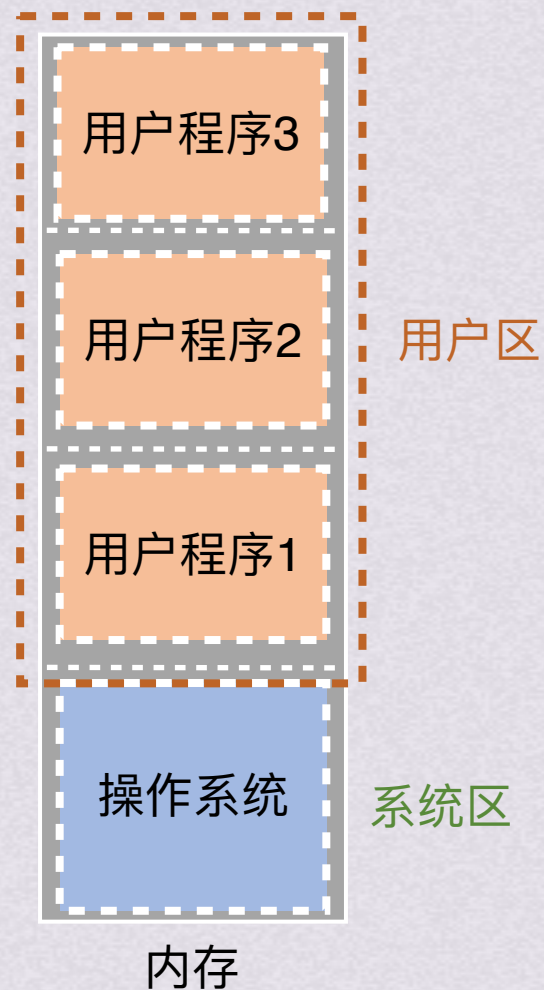


连续分配存储管理方式

固定分区分配

在后来出现的多道程序系统中，为了能在内存中装入多道程序，并且他们之间不会发生互相干扰，就把整个用户空间分成几个固定大小区域，每个分区只装一个作业

划分分区也可以划分成大小不同的分区



{ 单一连续分配
{ 动态分区分配

{ 离散分配存储管理方式

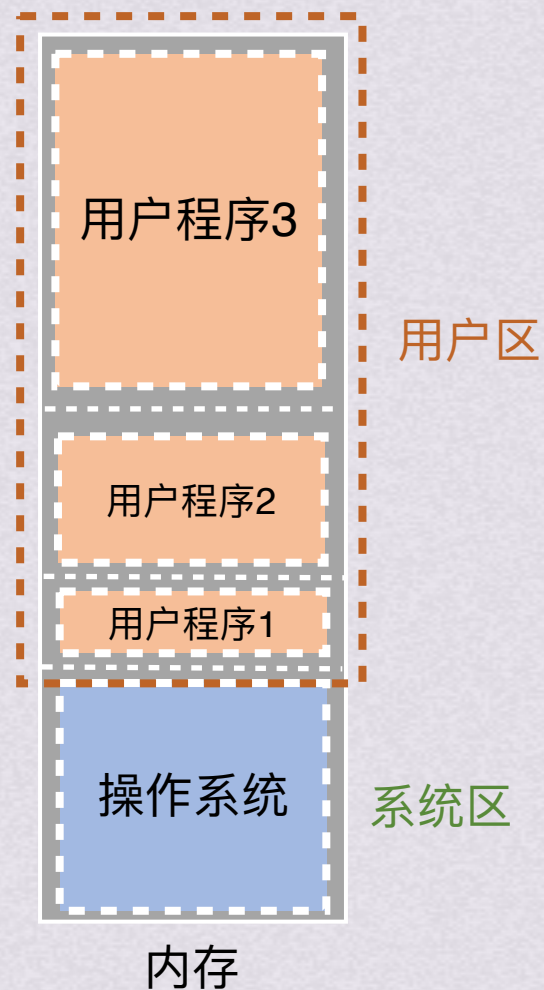


连续分配存储管理方式

固定分区分配

在后来出现的多道程序系统中，为了能在内存中装入多道程序，并且他们之间不会发生互相干扰，就把整个用户空间分成几个固定大小区域，每个分区只装一个作业

划分分区也可以划分成大小不同的分区



{ 单一连续分配
{ 动态分区分配

{ 离散分配存储管理方式



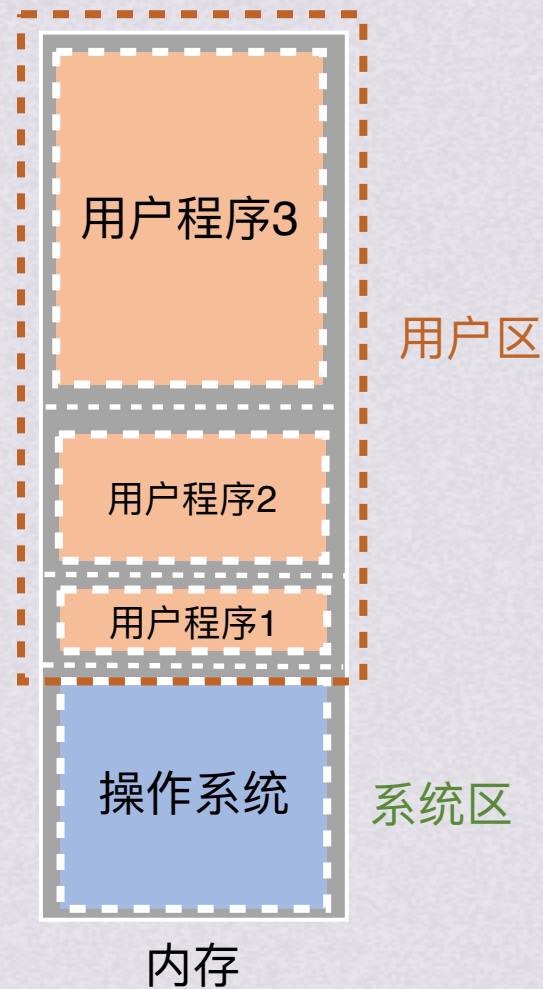
连续分配存储管理方式

固定分区分配

在后来出现的多道程序系统中，为了能在内存中装入多道程序，并且他们之间不会发生互相干扰，就把整个用户空间分成几个固定大小区域，每个分区只装一个作业

为了便于分配和回收，建立一张分区使用表，按分区大小排队，各表项包括对应分区的起始地址、大小、状态

分区号	大小/KB	起始地址/KB	状态
1	1	20	已分配
2	4	21	已分配
3	32	25	已分配

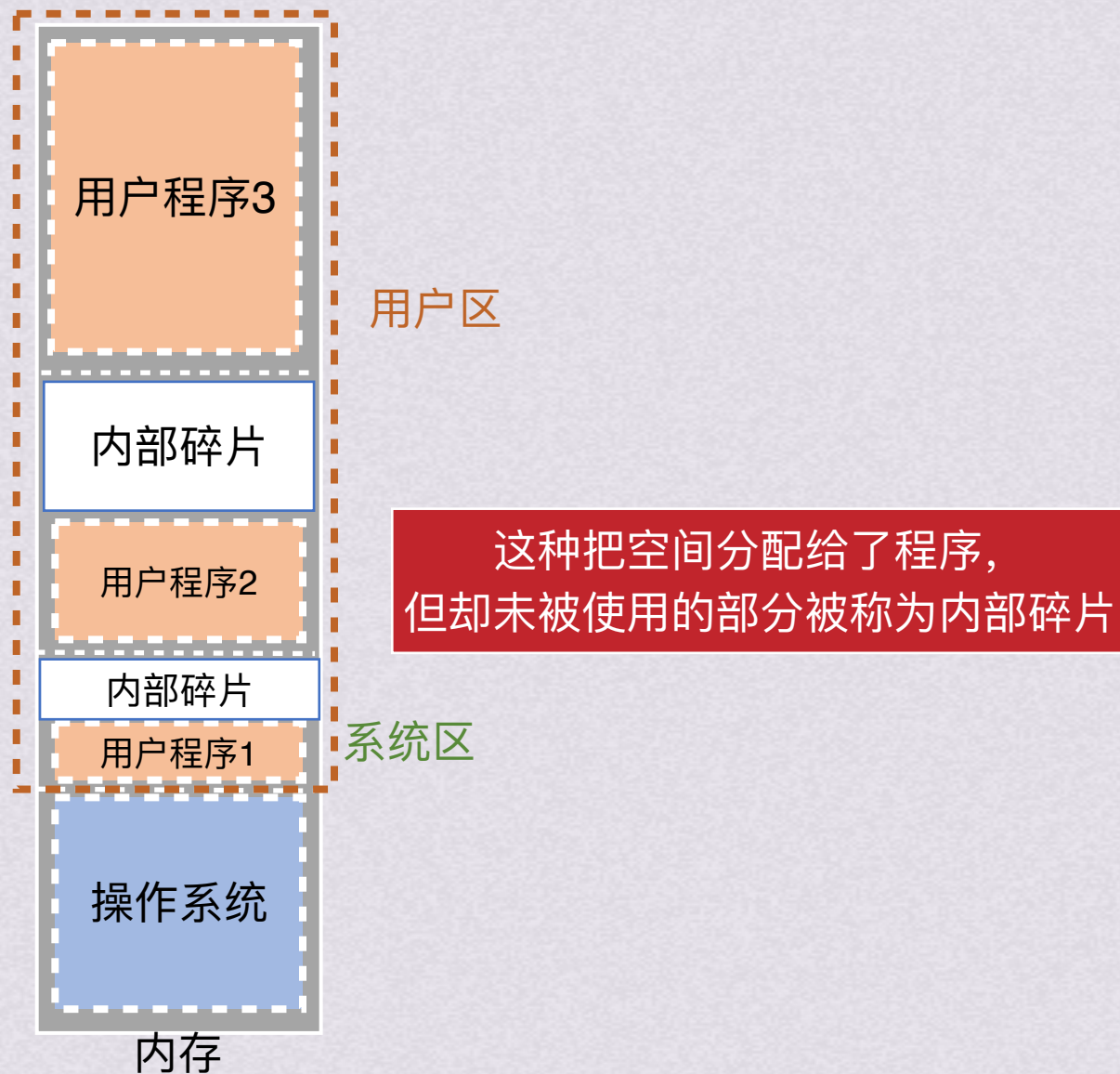




连续分配存储管理方式

固定分区分配

在后来出现的多道程序系统中，为了能在内存中装入多道程序，并且他们之间不会发生互相干扰，就把整个用户空间分成几个固定大小区域，每个分区只装一个作业



{ 单一连续分配
{ 动态分区分配

{ 离散分配存储管理方式



连续分配存储管理方式

固定分区分配

{ 单一连续分配
动态分区分配

{ 离散分配存储管理方式

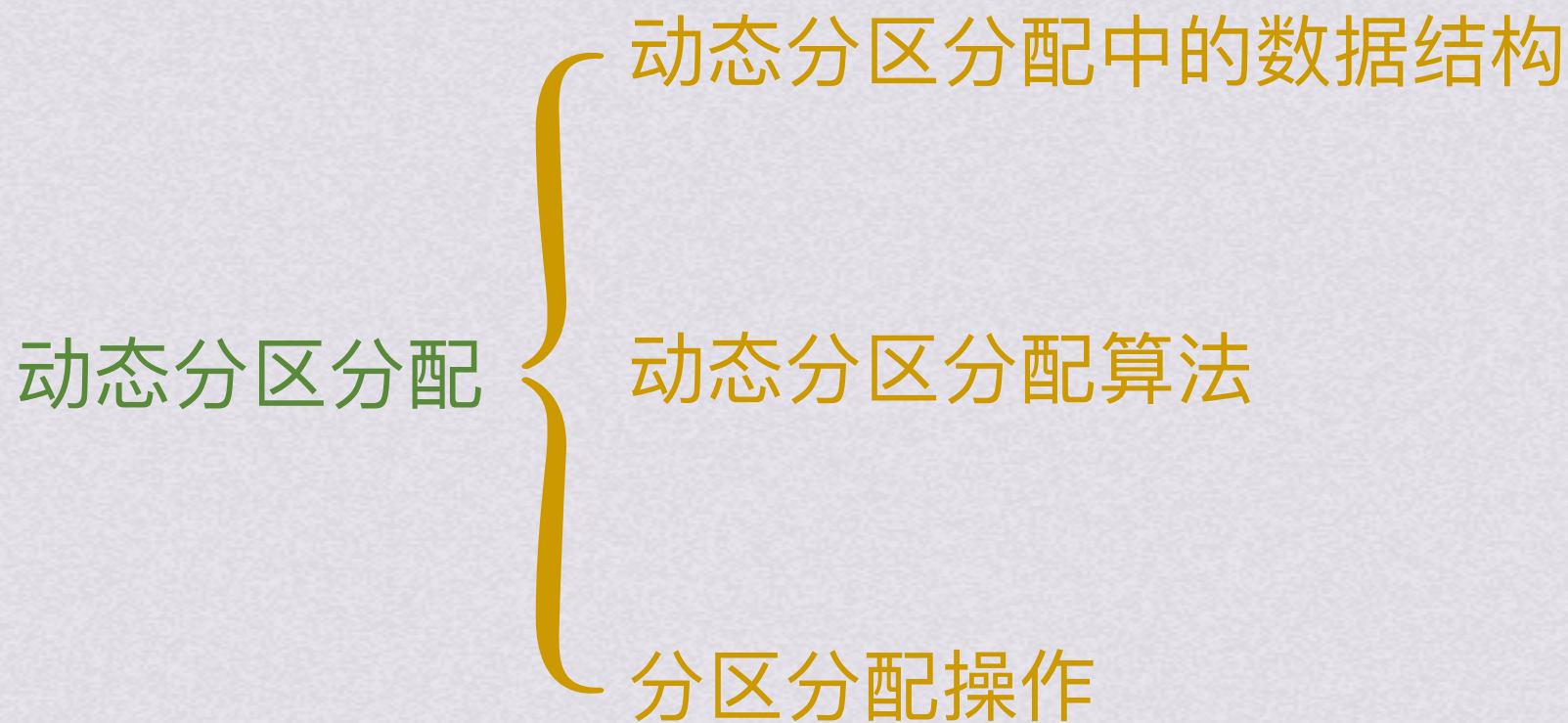


连续分配存储管理方式

动态分区分配



连续分配存储管理方式



动态分区分配

动态分区分配中的数据结构

实现动态分区分配需要配置对应的数据结构，用来描述空闲分区和已分配分区的情况

空闲分区表

系统中设置一张空闲分区表，记录每个空闲分区的情况。每个空闲分区占用一个表目，表目中包括分区号、分区大小和分区起始地址等数据

分区号	分区大小(KB)	分区始址	状态
1	50	85	空闲
2	32	155	空闲
3	70	275	空闲
4	60	532	空闲
5	...	...	...

{ 动态分区分配算法

{ 分区分配操作

{ 单一连续分配

{ 固定分区分配

{ 离散分配存储管理方式



动态分区分配

动态分区分配中的数据结构

- { 动态分区分配算法
- { 分区分配操作
- { 单一连续分配
- { 固定分区分配
- { 离散分配存储管理方式



动态分区分配

动态分区分配算法

- { 动态分区分配中的数据结构
 - { 分区分配操作
 - { 单一连续分配
 - { 固定分区分配
- { 离散分配存储管理方式



动态分区分配

动态分区分配算法

首次适应算法

邻近适应算法

最佳适应算法

最坏适应算法

{ 动态分区分配中的数据结构

{ 分区分配操作

{ 单一连续分配

{ 固定分区分配

{ 离散分配存储管理方式

动态分区分配算法

首次适应算法

空闲分区按照地址递增次序排列，每次分配内存，顺序查找第一个能满足大小的空闲分区，分配给作业

新作业

新作业大小35KB



已被占用

空闲

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	40	1140	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

- 邻近适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

首次适应算法

空闲分区按照地址递增次序排列，每次分配内存，顺序查找第一个能满足大小的空闲分区，分配给作业

新作业

新作业大小35KB



已被占用
空闲

内存

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	40	1140	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

- 邻近适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

首次适应算法

空闲分区按照地址递增次序排列，每次分配内存，顺序查找第一个能满足大小的空闲分区，分配给作业



已被占用
空闲

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	40	1140	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

- 邻近适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

首次适应算法

空闲分区按照地址递增次序排列，每次分配内存，顺序查找第一个能满足大小的空闲分区，分配给作业



已被占用
空闲

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3			空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

- 邻近适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

首次适应算法

空闲分区按照地址递增次序排列，每次分配内存，顺序查找第一个能满足大小的空闲分区，分配给作业



已被占用
空闲

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1175	空闲
4	5	1185	空闲
5	40	1192	空闲
6	20	1600	空闲

- 邻近适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配算法

首次适应算法

- { 邻近适应算法
- { 最佳适应算法
- { 最坏适应算法
- { 动态分区分配中的数据结构
- { 分区分配操作
- { 单一连续分配
- { 固定分区分配
- { 离散分配存储管理方式



动态分区分配算法

邻近适应算法 (循环首次适应算法)

- 首次适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

邻近适应算法(循环首次适应算法)

相比于首次适应，改为分配内存时从上次查找结束的位置开始继续查找

新作业

新作业大小15KB



已被占用
空闲

内存

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1185	空闲
4	5	1175	空闲
5	20	1600	空闲
6	40	1192	空闲

上次分配位置

- 首次适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

邻近适应算法(循环首次适应算法)

相比于首次适应，改为分配内存时从上次查找结束的位置开始继续查找

新作业 新作业大小15KB



已被占用
空闲

内存

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1185	空闲
4	5	1175	空闲
5	20	1600	空闲
6	40	1192	空闲

上次分配位置

首次适应算法
最佳适应算法
最坏适应算法
动态分区分配中的数据结构
分区分配操作
单一连续分配
固定分区分配
离散分配存储管理方式

动态分区分配算法

邻近适应算法(循环首次适应算法)

相比于首次适应，改为分配内存时从上次查找结束的位置开始继续查找



动态分区分配算法

邻近适应算法(循环首次适应算法)

相比于首次适应，改为分配内存时从上次查找结束的位置开始继续查找



20KB
已被占用
空闲

内存

分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	30	1106	空闲
3	5	1185	空闲
4	5	1175	空闲
5	5	1615	空闲
6	40	1192	空闲

上次分配位置

- 首次适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配算法

邻近适应算法 (循环首次适应算法)

- 首次适应算法
- 最佳适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配算法

最佳适应算法

- 首次适应算法
- 邻近适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

最佳适应算法

空闲分区按照容量递增排列，每次分配内存的时候找到最小的且能满足大小的空闲分区

新作业

新作业大小18KB



已被占用

空闲

内存

分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	20	1600	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

- 首次适应算法
- 邻近适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

最佳适应算法

空闲分区按照容量递增排列，每次分配内存的时候找到最小的且能满足大小的空闲分区

新作业

新作业大小18KB



已被占用

空闲

内存

分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	20	1600	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

- 首次适应算法
- 邻近适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

最佳适应算法

空闲分区按照容量递增排列，每次分配内存的时候找到最小的且能满足大小的空闲分区



内存

分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	20	1600	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

首次适应算法
邻近适应算法
最坏适应算法
动态分区分配中的数据结构
分区分配操作
单一连续分配
固定分区分配
离散分配存储管理方式



动态分区分配算法

最佳适应算法

空闲分区按照容量递增排列，每次分配内存的时候找到最小的且能满足大小的空闲分区



15KB

内存

20KB
已被占用
空闲

分区号	分区大小/KB	分区始址/KB	状态
1	5	1185	空闲
2	5	1615	空闲
3	30	1070	空闲
4	30	1106	空闲
5	40	1140	空闲
6	40	1192	空闲

如果有新的空闲分区的话需要重新按照大小排序

首次适应算法
邻近适应算法
最坏适应算法
动态分区分配中的数据结构
分区分配操作
单一连续分配
固定分区分配
离散分配存储管理方式



动态分区分配算法

最佳适应算法

- 首次适应算法
- 邻近适应算法
- 最坏适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配算法

最坏适应算法

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

最坏适应算法

空闲分区按照容量递减的次序排列，每次分配内存的时候找最大的空闲分区分配

新作业

新作业大小5KB



分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	40	1192	空闲
3	30	1070	空闲
4	30	1106	空闲
5	20	1600	空闲
6	5	1185	空闲

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

最坏适应算法

空闲分区按照容量递减的次序排列，每次分配内存的时候找最大的空闲分区分配



分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	40	1192	空闲
3	30	1070	空闲
4	30	1106	空闲
5	20	1600	空闲
6	5	1185	空闲

调整空闲分区表

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

最坏适应算法

空闲分区按照容量递减的次序排列，每次分配内存的时候找最大的空闲分区分配



已被占用
空闲

内存				5KB
分区号	分区大小/KB	分区始址/KB	状态	调整空闲分区表 新分区需要排序
1	35	1145	空闲	
2	40	1192	空闲	
3	30	1070	空闲	
4	30	1106	空闲	
5	20	1600	空闲	
6	5	1185	空闲	

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

动态分区分配算法

最坏适应算法

空闲分区按照容量递减的次序排列，每次分配内存的时候找最大的空闲分区分配



已被占用
空闲

分区号	分区大小/KB	分区始址/KB	状态
1	40	1192	空闲
2	35	1145	空闲
3	30	1070	空闲
4	30	1106	空闲
5	20	1600	空闲
6	5	1185	空闲

新分区需要排序

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式

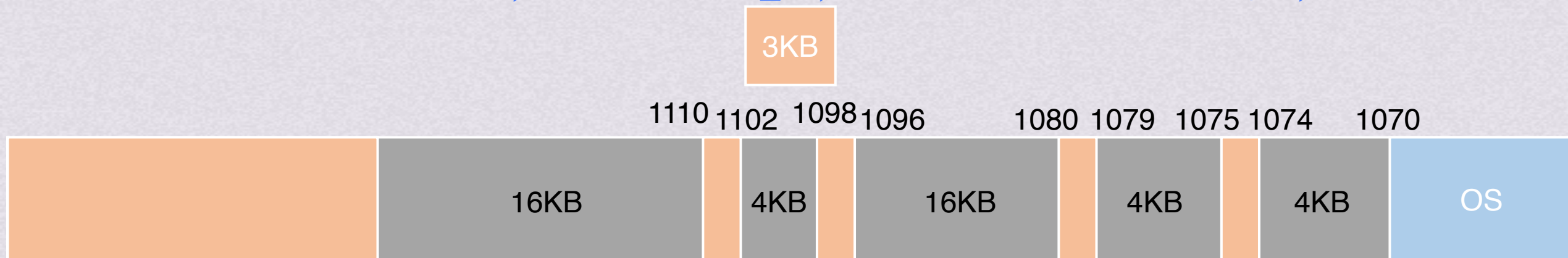


动态分区分配算法

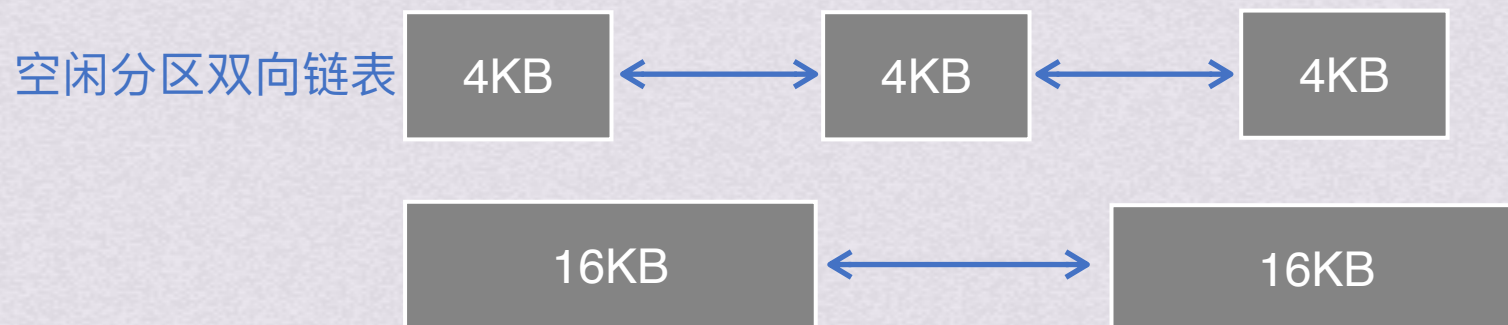
伙伴系统

伙伴系统规定：所有分区均为2的k次幂，k是正整数。所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表

需要给进程分配大小为n的空间时，先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配



已被占用
空闲



首次适应算法
邻近适应算法
最佳适应算法
动态分区分配中的数据结构
分区分配操作
单一连续分配
固定分区分配
离散分配存储管理方式

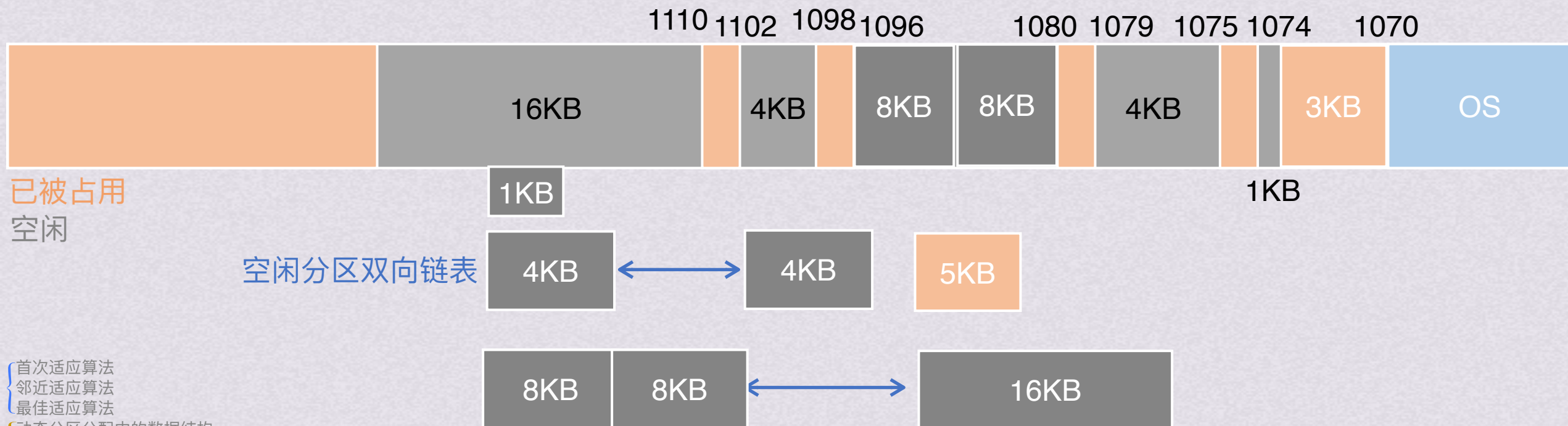


动态分区分配算法

伙伴系统

伙伴系统规定：所有分区均为2的k次幂，k是正整数。所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表

需要给进程分配大小为n的空间时，先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配
找不到就在大小 2^{i+1} 的空闲分区链表中找，如果找到，把该分区分成两个相同大小分区，称为一对伙伴
其中一个分区用于分配，再把另一个分区加入大小为 2^i 的空闲分区链表中



8KB分区已用尽，把16KB分区拆分成两个8KB再分配

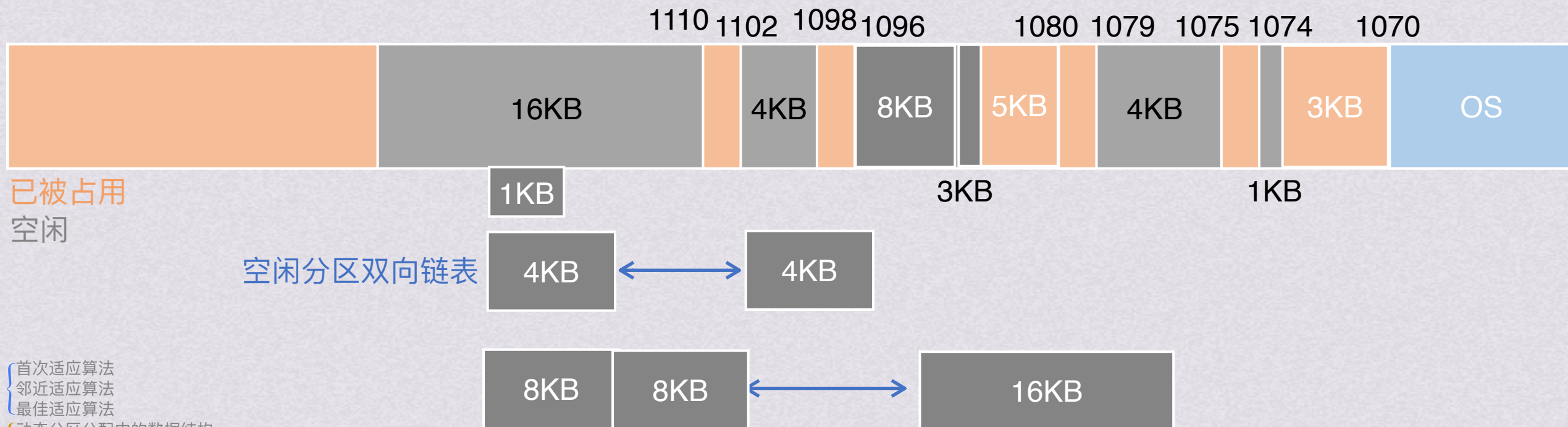


动态分区分配算法

伙伴系统

伙伴系统规定：所有分区均为2的k次幂，k是正整数。所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表

需要给进程分配大小为n的空间时，先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配
找不到就在大小 2^{i+1} 的空闲分区链表中找，如果找到，把该分区分成两个相同大小分区，称为一对伙伴
其中一个分区用于分配，再把另一个分区加入大小为 2^i 的空闲分区链表中



首次适应算法
邻近适应算法
最佳适应算法
动态分区分配中的数据结构
分区分配操作
单一连续分配
固定分区分配

离散分配存储管理方式

8KB分区已用尽，把16KB分区拆分成两个8KB再分配

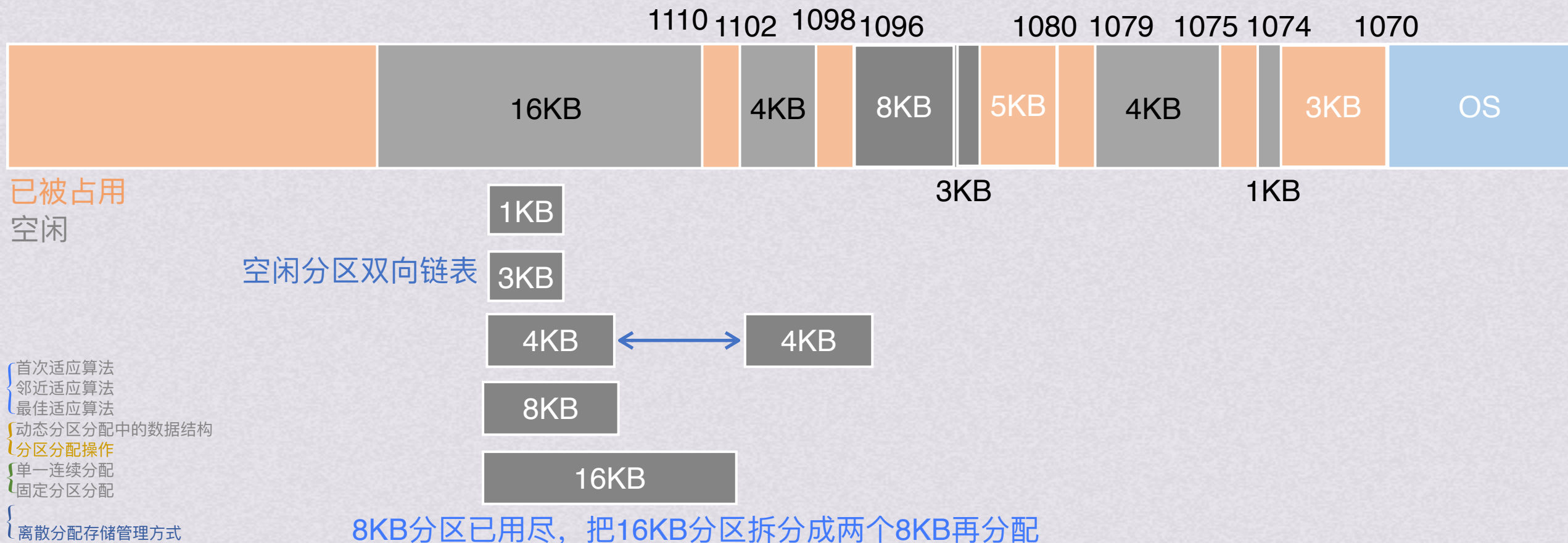


动态分区分配算法

伙伴系统

伙伴系统规定：所有分区均为2的k次幂，k是正整数。所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表

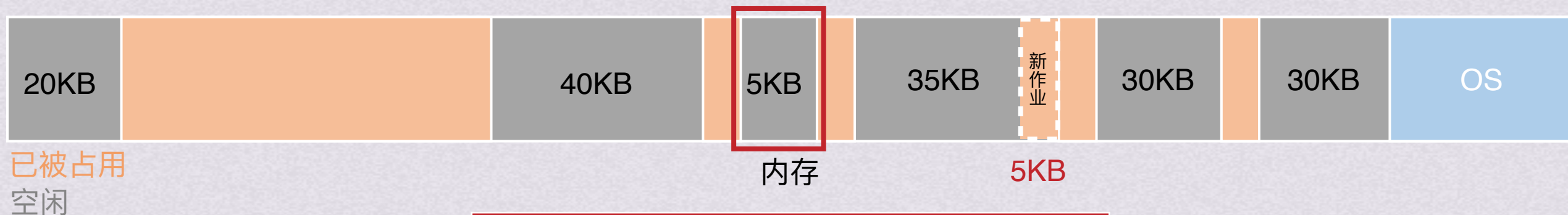
需要给进程分配大小为n的空间时，先计算 $2^{i-1} < n \leq 2^i$ ，在分区大小为 2^i 的空闲分区链表中找，找到就分配
找不到就在大小 2^{i+1} 的空闲分区链表中找，如果找到，把该分区分成两个相同大小分区，称为一对伙伴
其中一个分区用于分配，再把另一个分区加入大小为 2^i 的空闲分区链表中





动态分区分配算法

紧凑技术



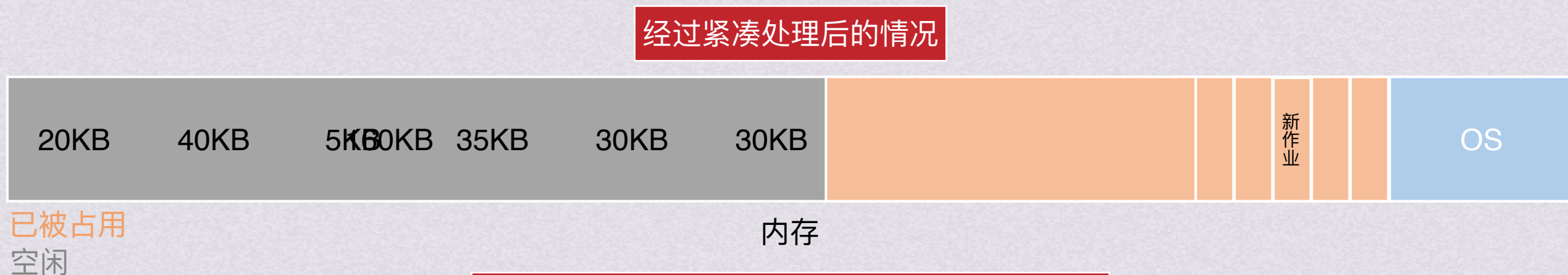
这种小的空间难以被分配出去，被称为外部碎片
可以通过紧凑技术来解决
但是需要动态重定位寄存器的支持，而且费时

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配算法

紧凑技术



这种小的空间难以被分配出去，被称为外部碎片
可以通过紧凑技术来解决
但是需要动态重定位寄存器的支持，而且费时

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配算法

最坏适应算法

- 首次适应算法
- 邻近适应算法
- 最佳适应算法
- 动态分区分配中的数据结构
- 分区分配操作
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配

动态分区分配算法

首次适应算法

邻近适应算法

最佳适应算法

最坏适应算法

{ 动态分区分配中的数据结构

{ 分区分配操作

{ 单一连续分配

{ 固定分区分配

{ 离散分配存储管理方式



动态分区分配

动态分区分配算法

- { 动态分区分配中的数据结构
 - { 分区分配操作
 - { 单一连续分配
 - { 固定分区分配
- { 离散分配存储管理方式



动态分区分配

分区分配操作

- { 动态分区分配中的数据结构
- { 动态分区分配算法
- { 单一连续分配
- { 固定分区分配
- { 离散分配存储管理方式

动态分区分配

分区分配操作

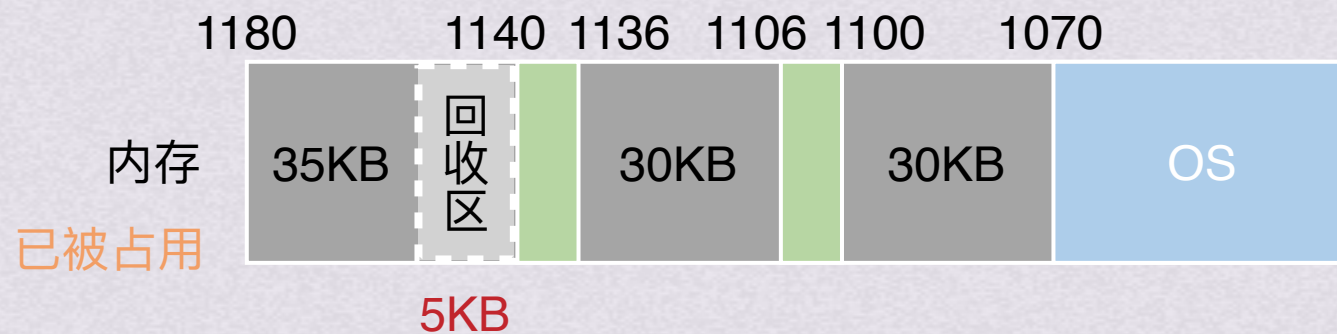
有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	35	1145	空闲
2	30	1070	空闲
3	30	1106	空闲

{ 动态分区分配中的数据结构
{ 动态分区分配算法
{ 单一连续分配
{ 固定分区分配
{ 离散分配存储管理方式

动态分区分配

分区分配操作

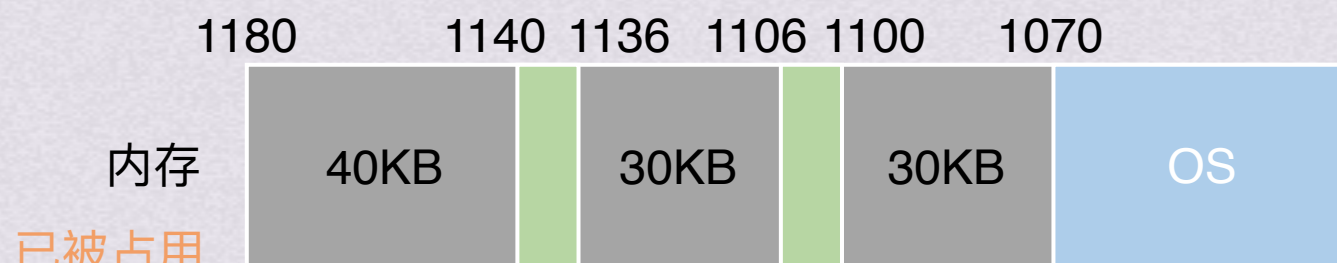
有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	35	1145	空闲
2	30	1070	空闲
3	30	1106	空闲

调整空闲分区表

动态分区分配

分区分配操作

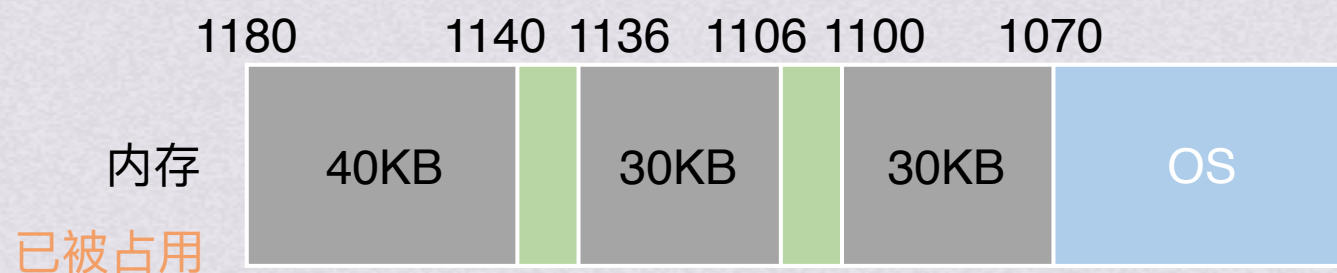
有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	30	1070	空闲
3	30	1106	空闲

调整空闲分区表



动态分区分配

分区分配操作

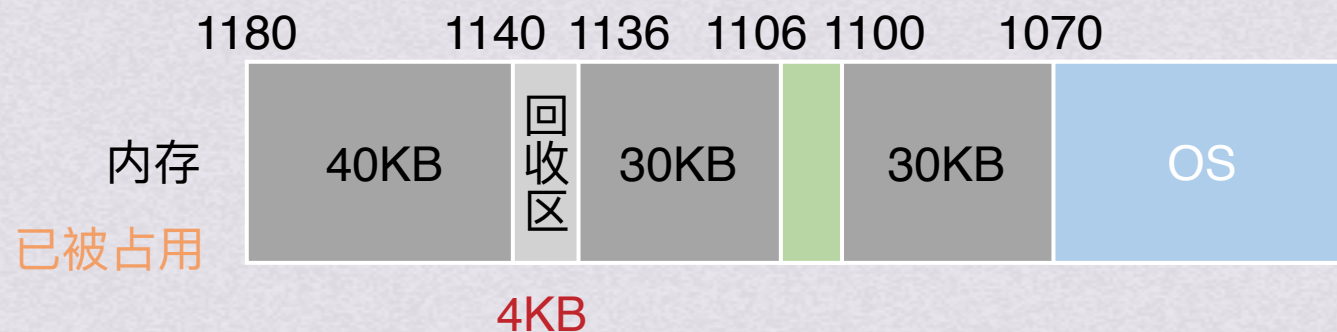
有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间 $<$ 空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	30	1070	空闲
3	30	1106	空闲

动态分区分配

分区分配操作

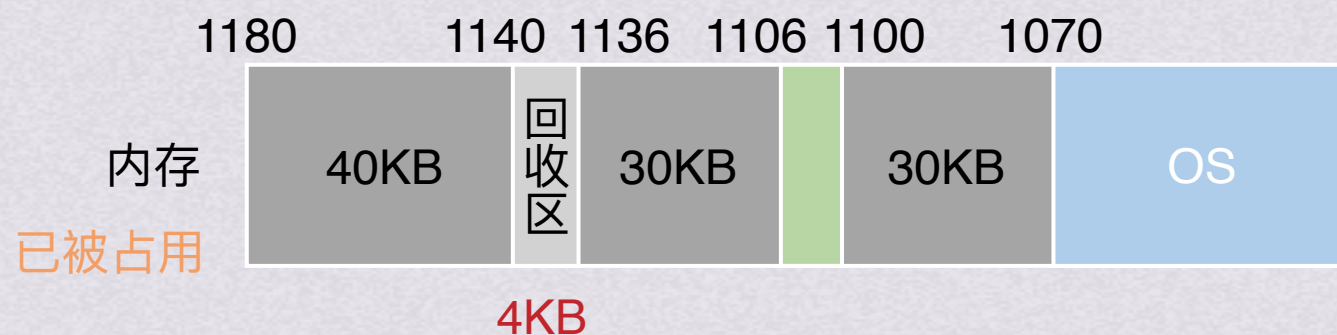
有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	30	1070	空闲
3	30	1106	空闲

{ 动态分区分配中的数据结构
{ 动态分区分配算法
{ 单一连续分配
{ 固定分区分配
{ 离散分配存储管理方式

动态分区分配

分区分配操作

有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	40	1140	空闲
2	30	1070	空闲
3	30	1106	空闲

调整空闲分区表

{ 动态分区分配中的数据结构
{ 动态分区分配算法
{ 单一连续分配
{ 固定分区分配
{ 离散分配存储管理方式

动态分区分配

分区分配操作

有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
2	30	1070	空闲
3	74	1106	空闲

调整空闲分区表

{ 动态分区分配中的数据结构
{ 动态分区分配算法
{ 单一连续分配
{ 固定分区分配
{ 离散分配存储管理方式



动态分区分配

分区分配操作

有分配和回收两个操作

分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并



分区号	分区大小/KB	分区始址/KB	状态
1	30	1070	空闲
2	74	1106	空闲

调整空闲分区表

- 动态分区分配中的数据结构
- 动态分区分配算法
- 单一连续分配
- 固定分区分配
- 离散分配存储管理方式



动态分区分配

分区分配操作

有分配和回收两个操作

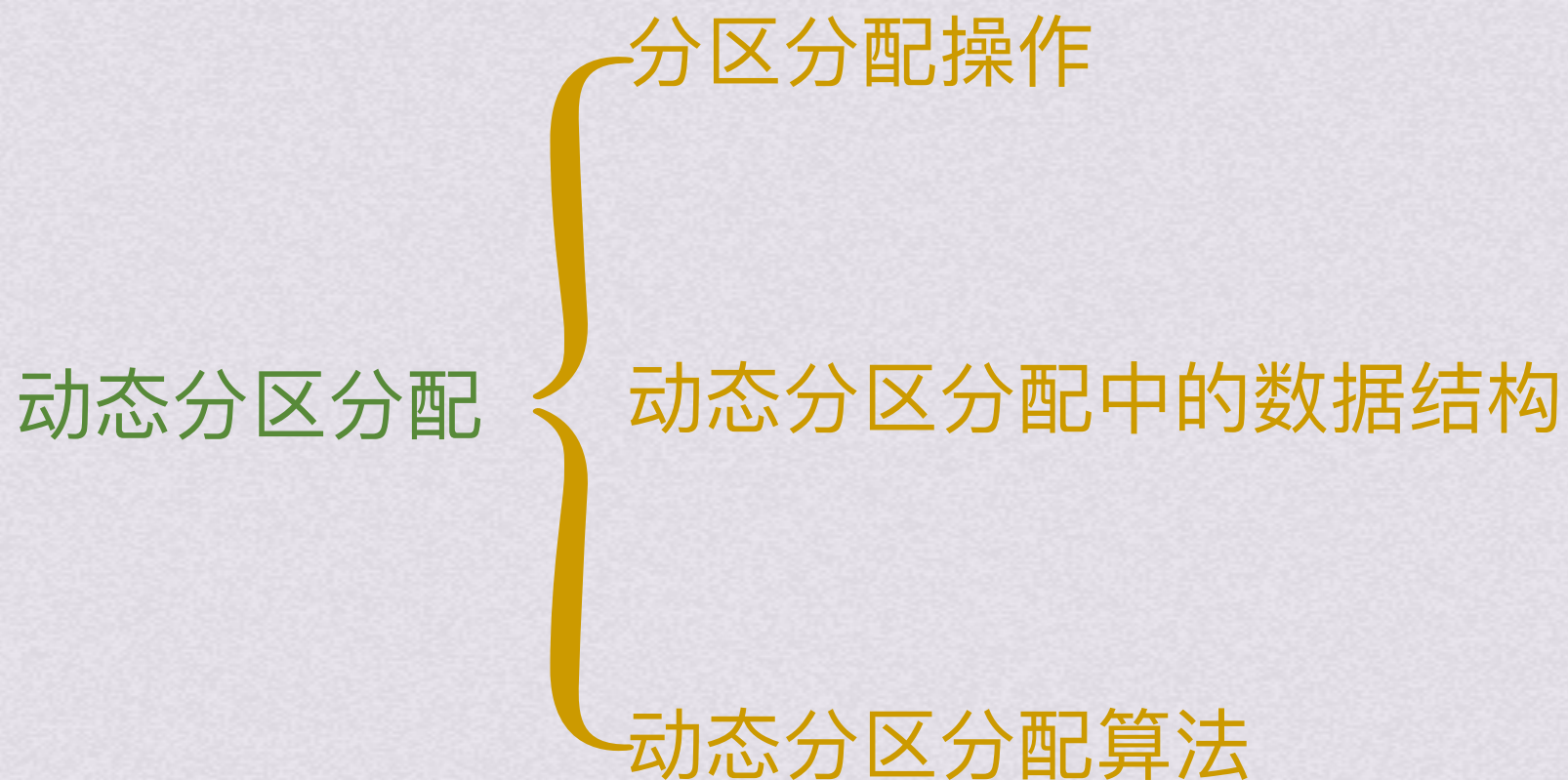
分配内存：

根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间 $<$ 空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序；其余分配给发出申请的进程

回收内存：

进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并

根据回收区是否和前一个或者后一个，或者和前后都相连，或者与前后都不相连
分别在空闲分区表中选择修改表项，或者合并表项，或者增加表项



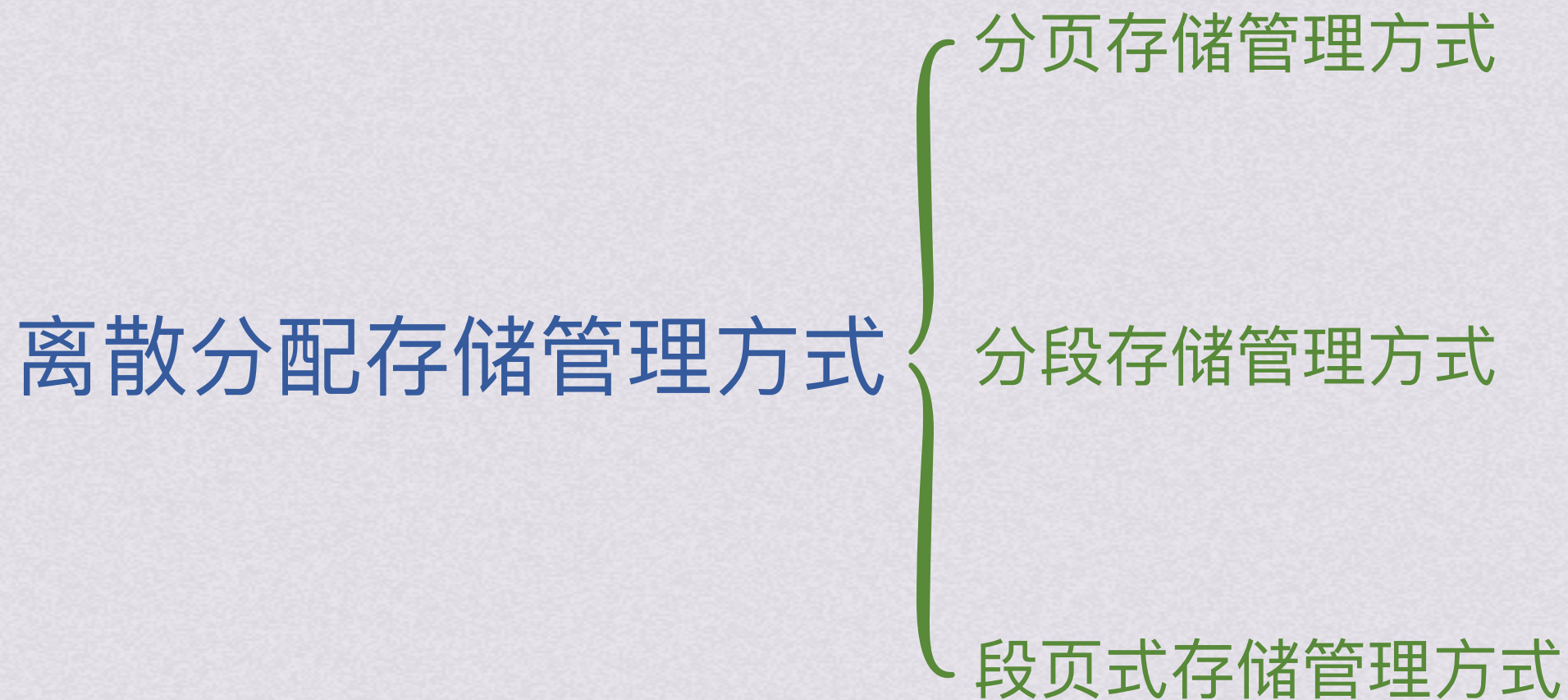




连续分配存储管理方式



离散分配存储管理方式





离散分配存储管理方式

分页存储管理方式



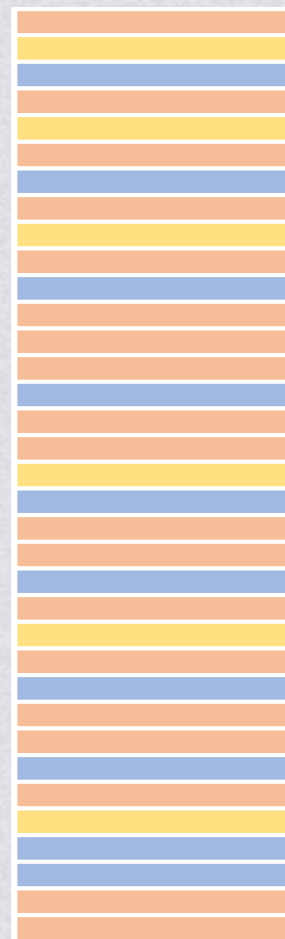
离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



连续分配下的内存



分页存储下的内存

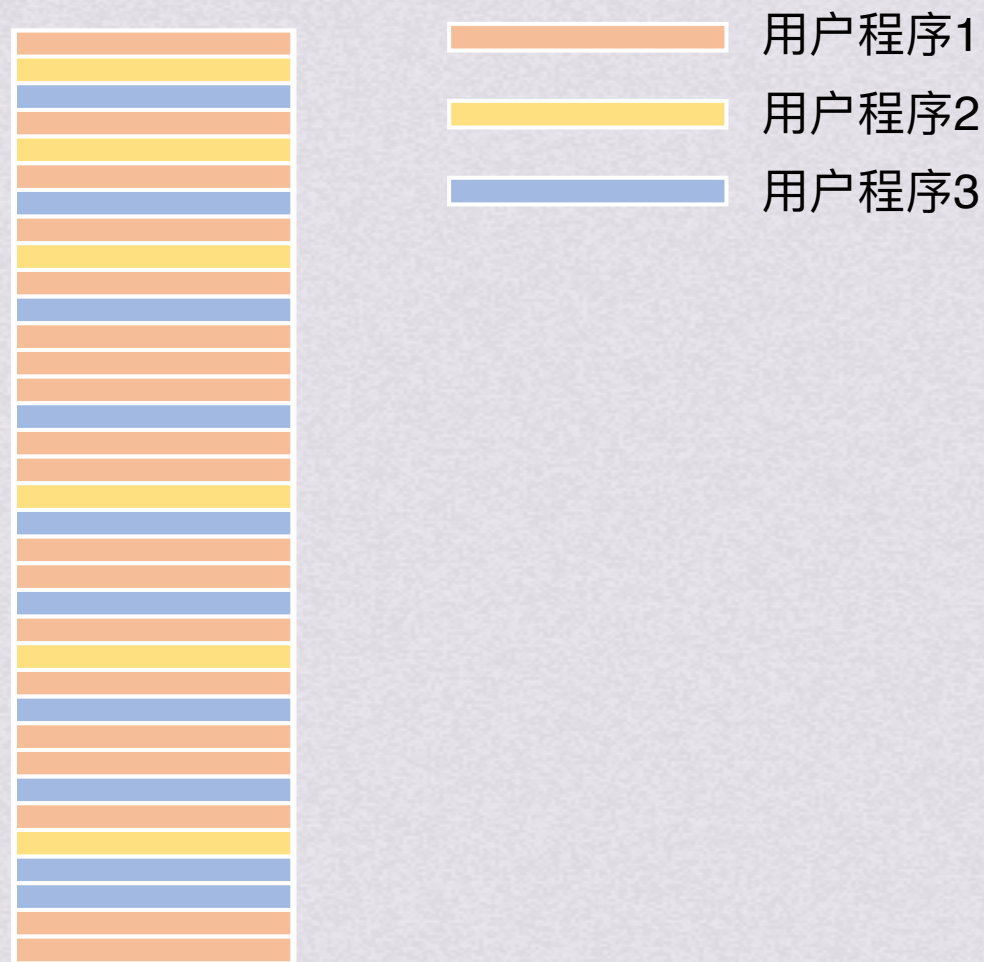




离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



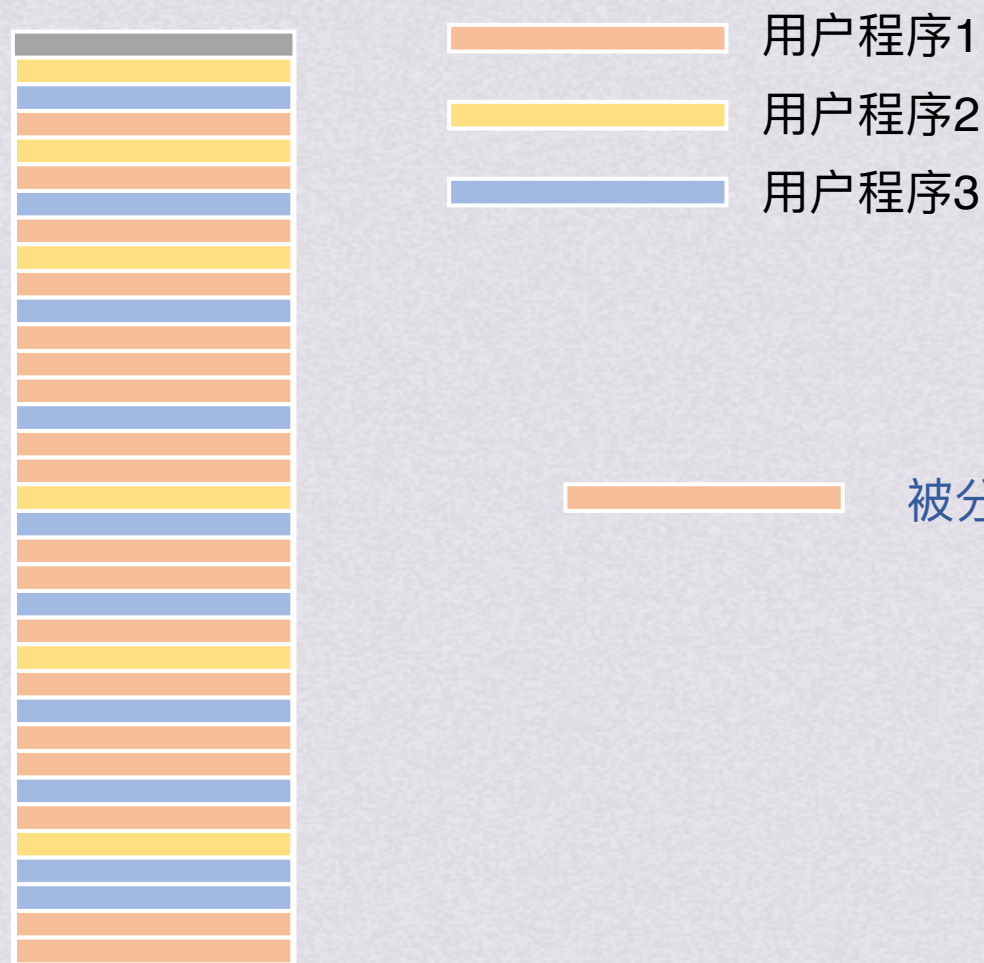
分页存储下的内存



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



被分割成的固定大小的区域被称为“页”或“页面”

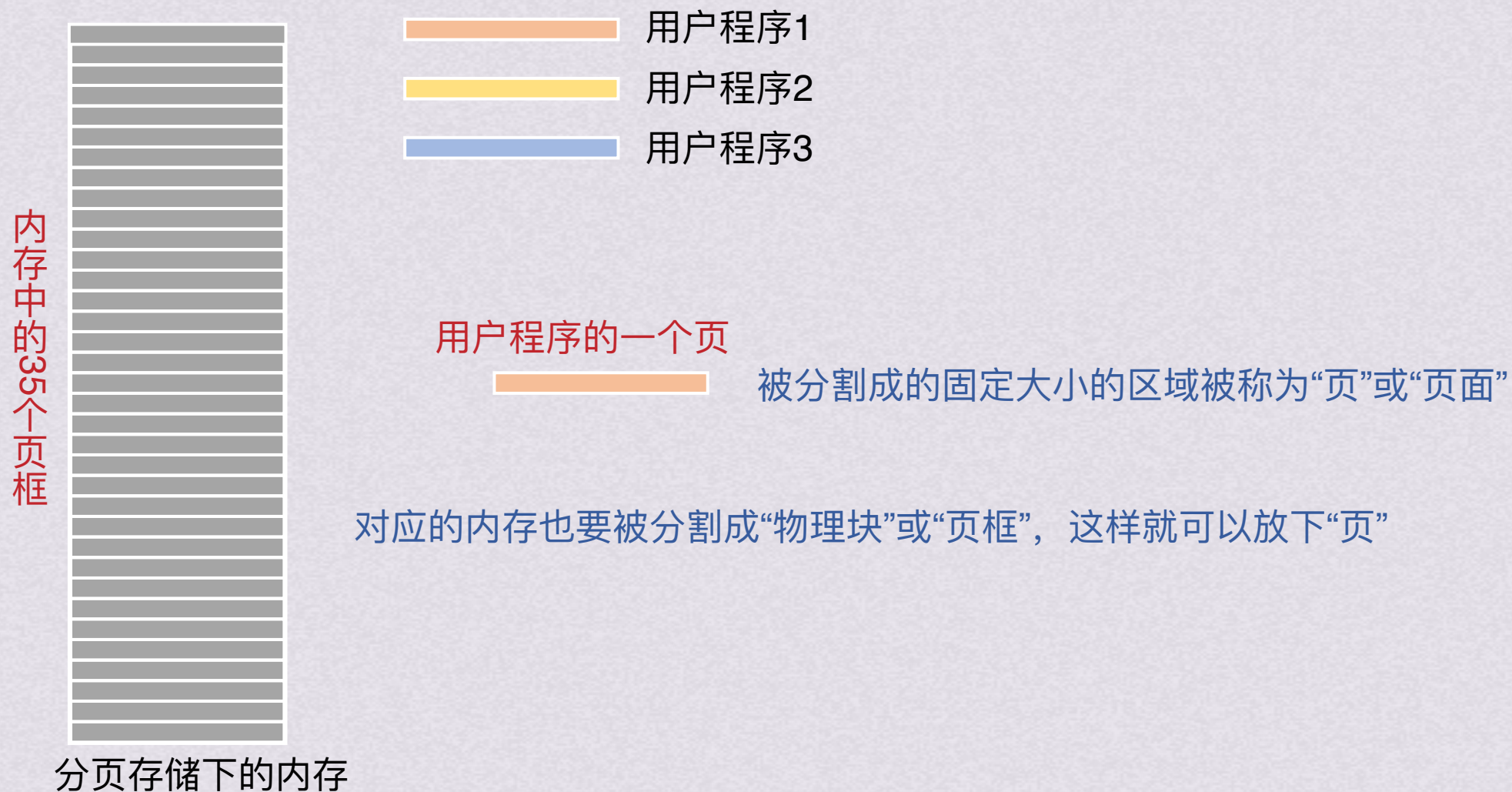
分页存储下的内存



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了

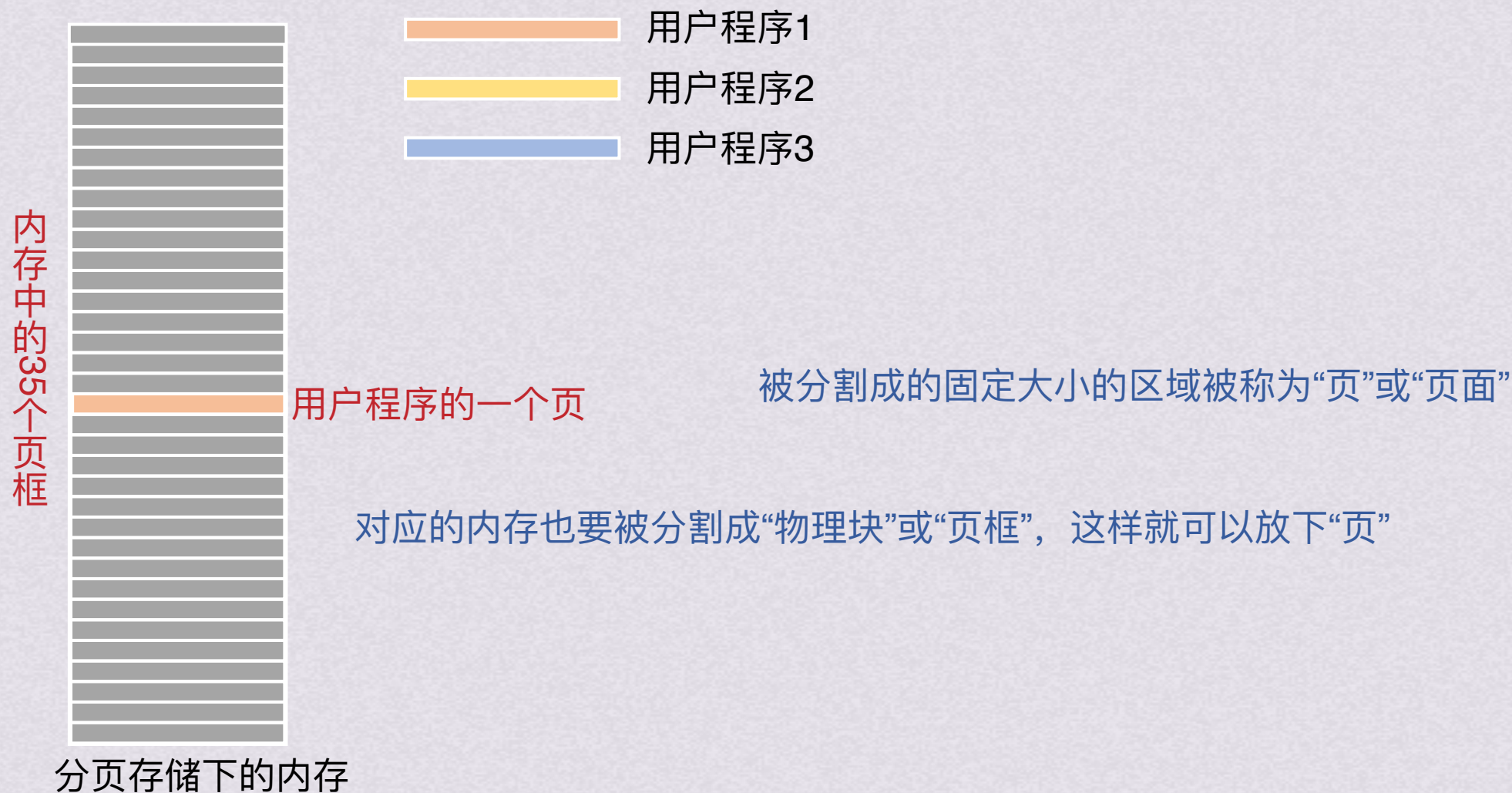




离散分配存储管理方式

分页存储管理方式

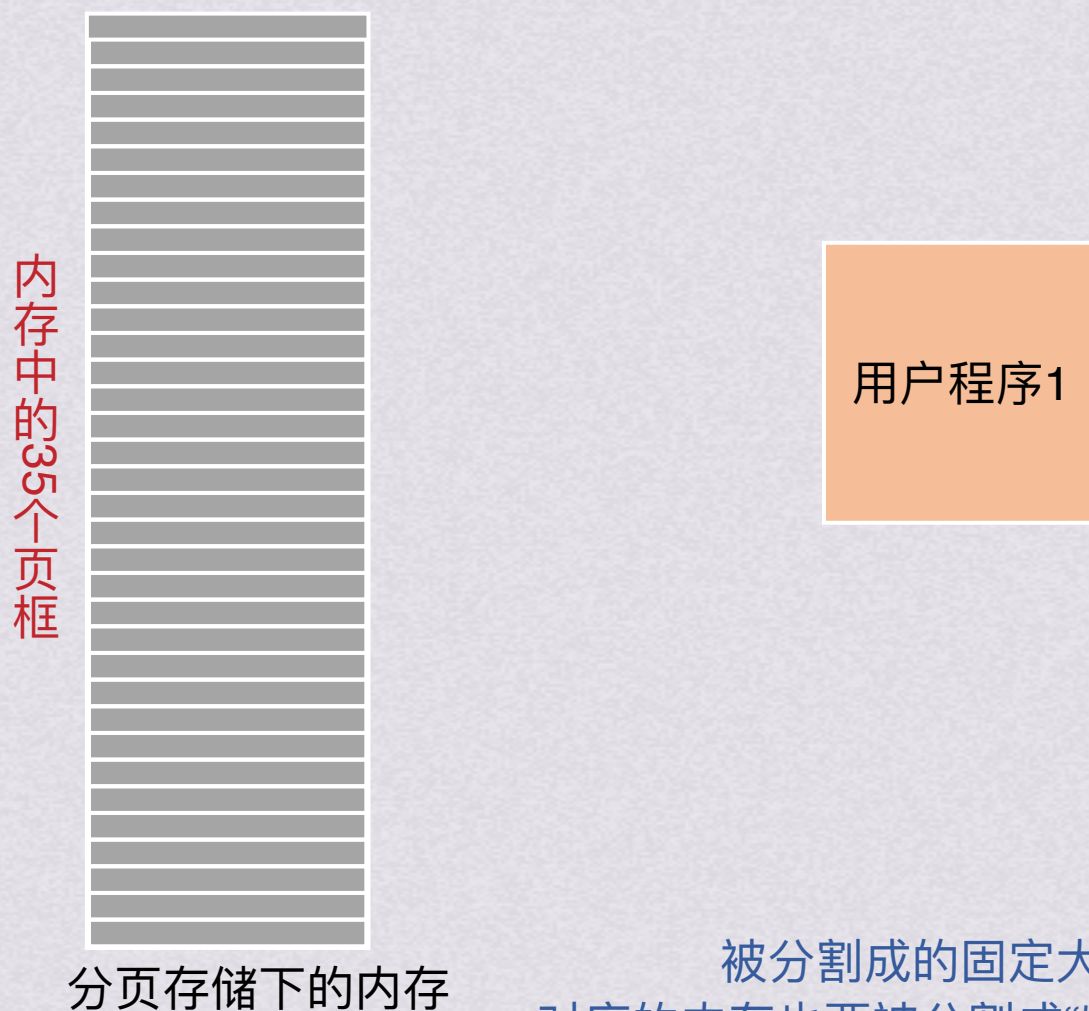
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了





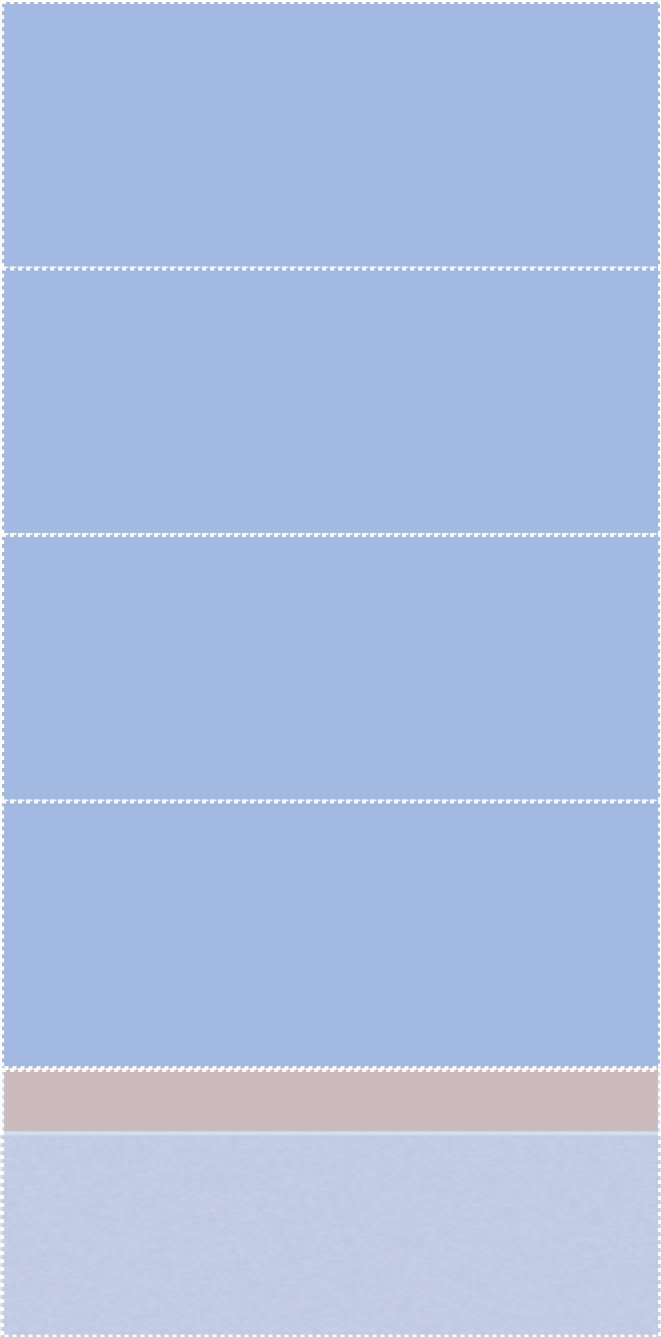
分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



被分割成的固定大小的区域被称为“页”或“页面”
对应的内存也要被分割成“物理块”或“页框”，这样就可以放下“页”

用户程序1





离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了





离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了





离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了





离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销
所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了

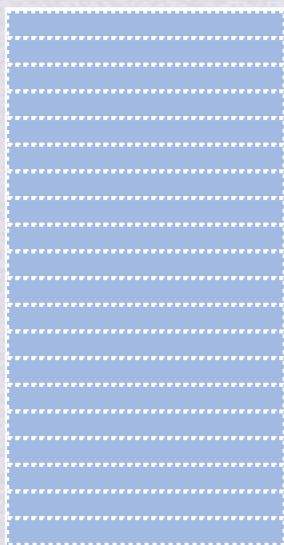




离散分配存储管理方式

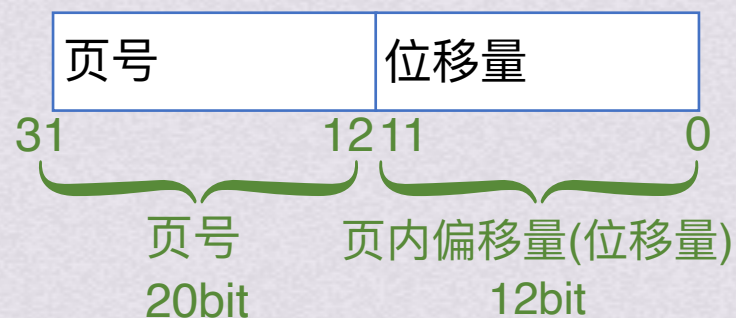
分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



用户程序1

分页地址中的地址结构（逻辑地址）
假设每页大小 $4KB = 2^{12}B$,共有 $1M = 2^{20}$ 页

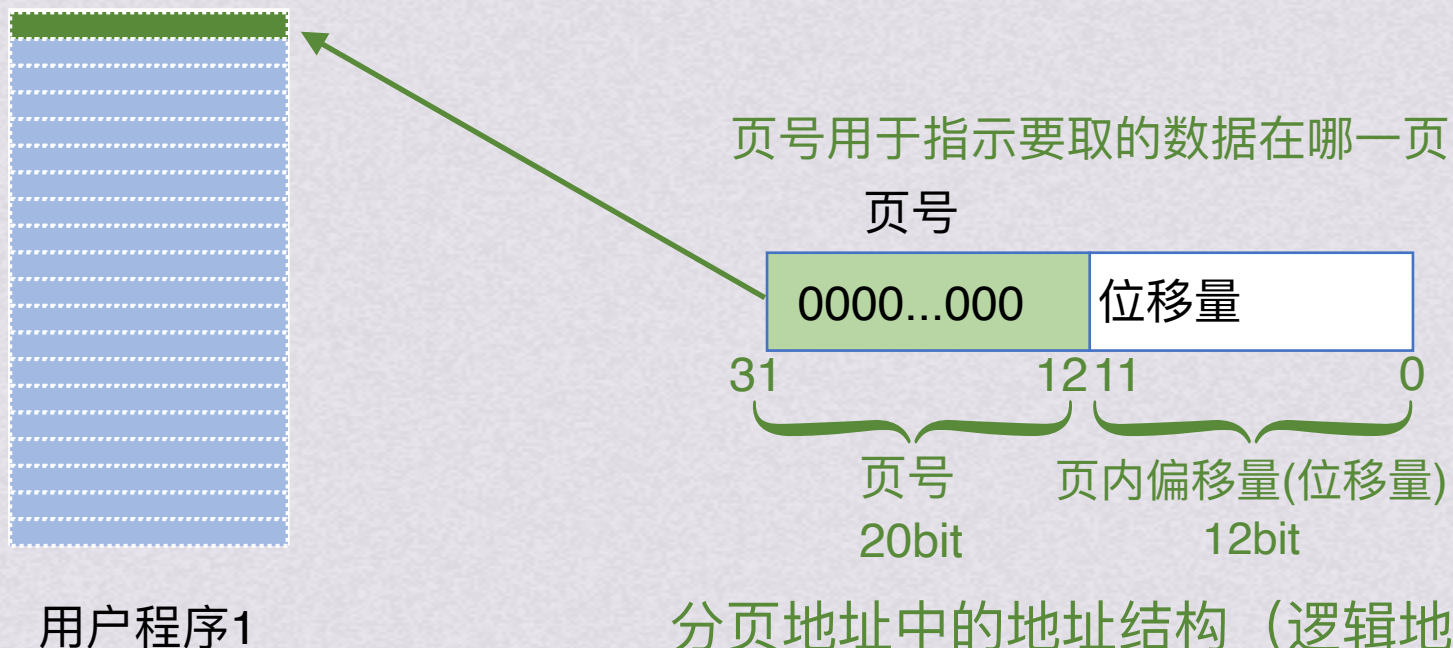




离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



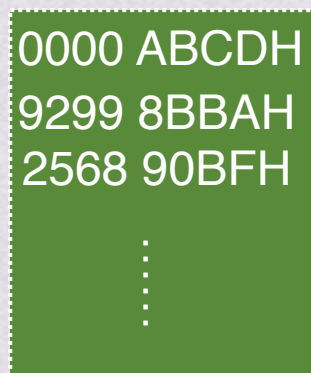
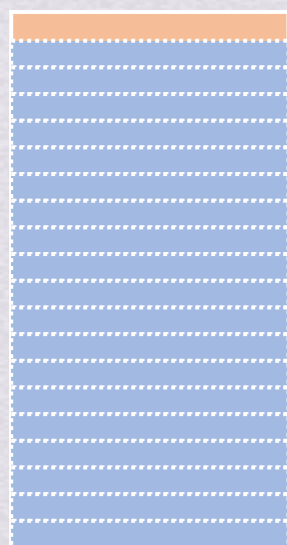
假设每页大小 $4KB = 2^{12}B$,共有 $1M = 2^{20}$ 页



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



页号用于指示要取的数据在哪一页

页号



用户程序1

分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$,共有 $1M = 2^{20}$ 页

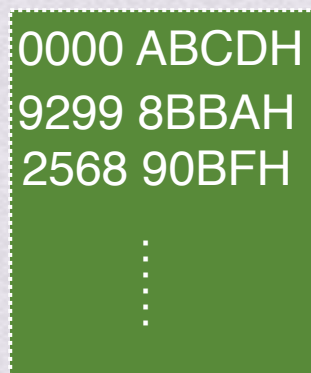
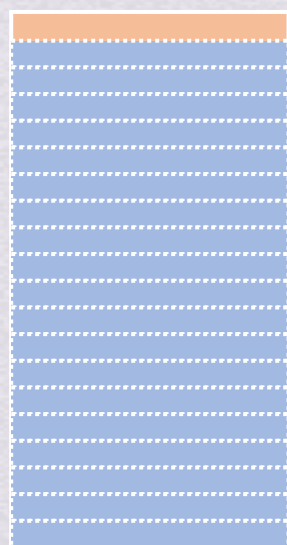
{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



页号用于指示要取的数据在哪一页

页号



分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$,共有 $1M = 2^{20}$ 页

用户程序1

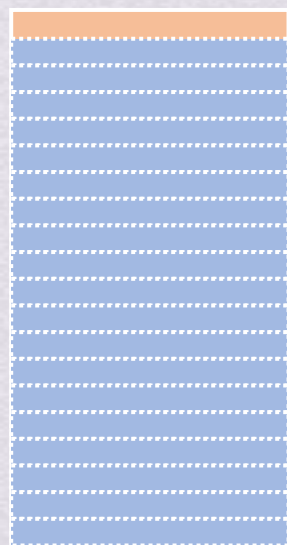
{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



用户程序1



页号用于指示要取的数据在哪一页
位移量用于指示取出这一页的哪一个存储单元



分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$,共有 $1M = 2^{20}$ 页

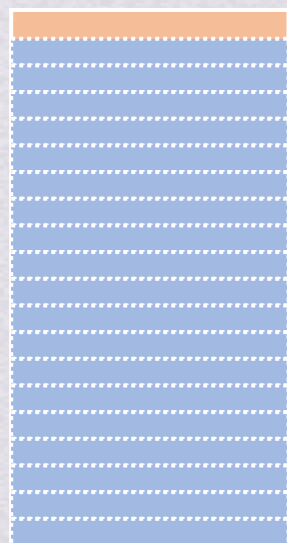
- 分段存储管理方式
- 段页式存储管理方式
- 连续分配存储管理方式



离散分配存储管理方式

分页存储管理方式

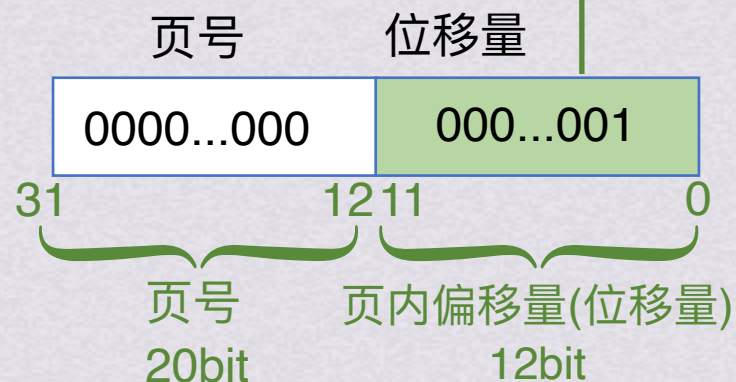
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



用户程序1



页号用于指示要取的数据在哪一页
位移量用于指示取出这一页的哪一个存储单元



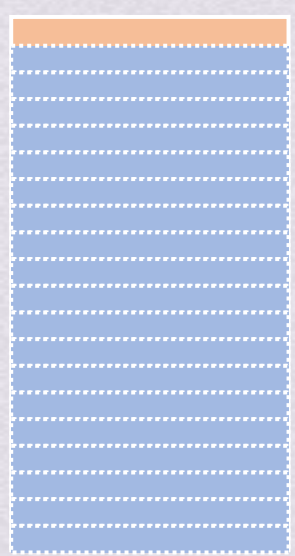
分页地址中的地址结构（逻辑地址）

假设每页大小 $4KB = 2^{12}B$,共有 $1M = 2^{20}$ 页

离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



用户程序1

- { 分段存储管理方式
- { 段页式存储管理方式
- { 连续分配存储管理方式

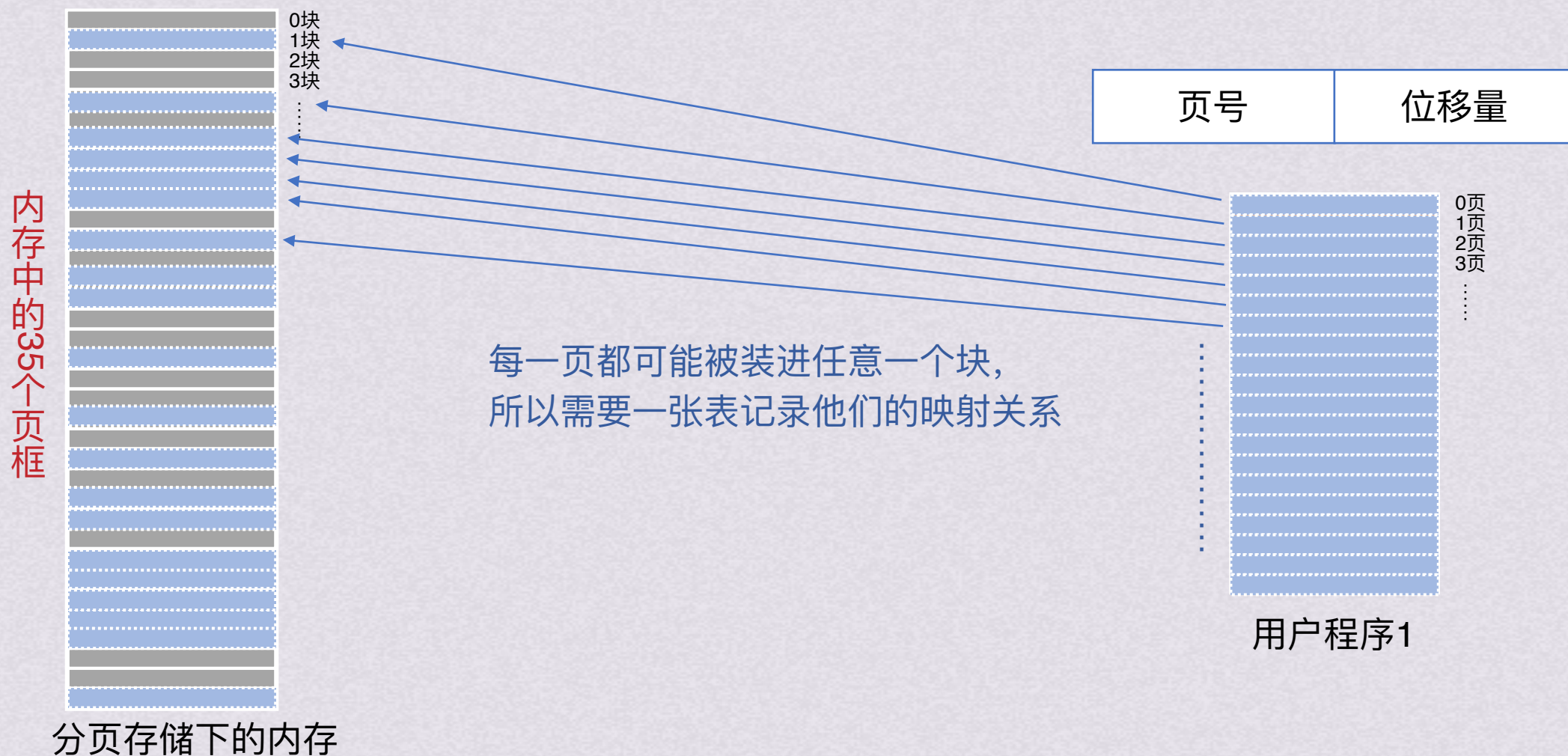
页号	位移量
----	-----



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



离散分配存储管理方式

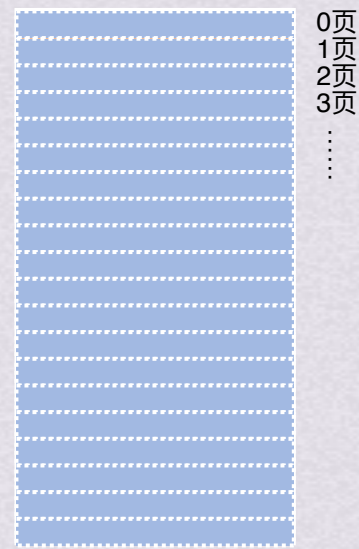
分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
.....	

页号	位移量
----	-----



系统为每个进程建立了一张页面映射表，简称**页表** 用户程序1
通过查找页表就可以知道程序的某一页存在内存中哪个块

分页存储下的内存 页表大多驻留在内存中，系统中设置一页表寄存器
其存放页表在内存中的起始地址和页表长度。进程未执行时这两个数据放在PCB中，执行时才装入寄存器

{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了

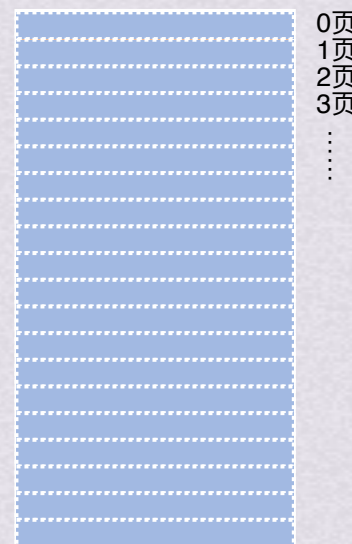


分页存储下的内存

页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
.....	

实际上页表通常不记录页号，把它隐含在了页表的结构中

页号	位移量
----	-----



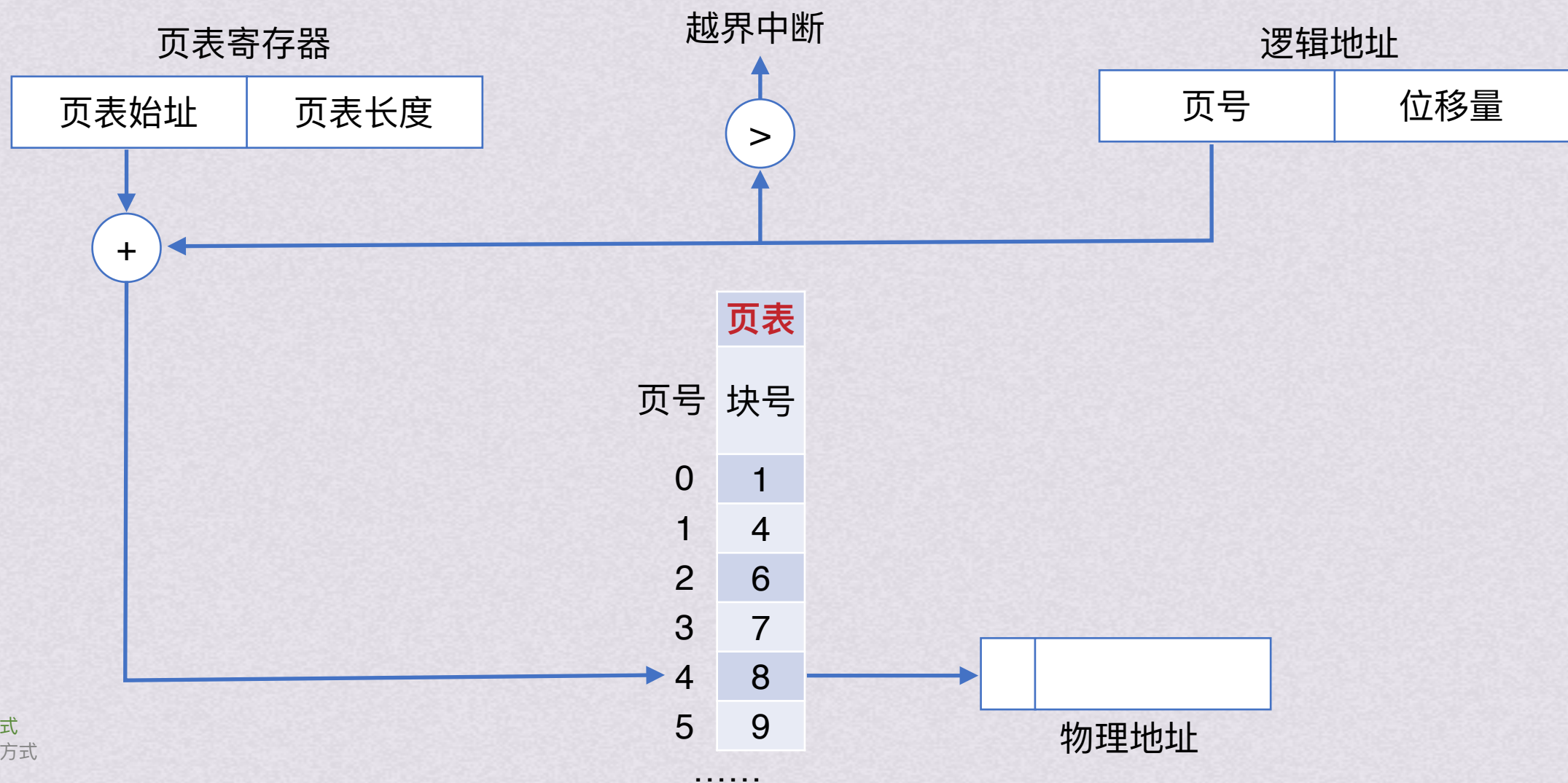
用户程序1

- 分段存储管理方式
- 段页式存储管理方式
- 连续分配存储管理方式

离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



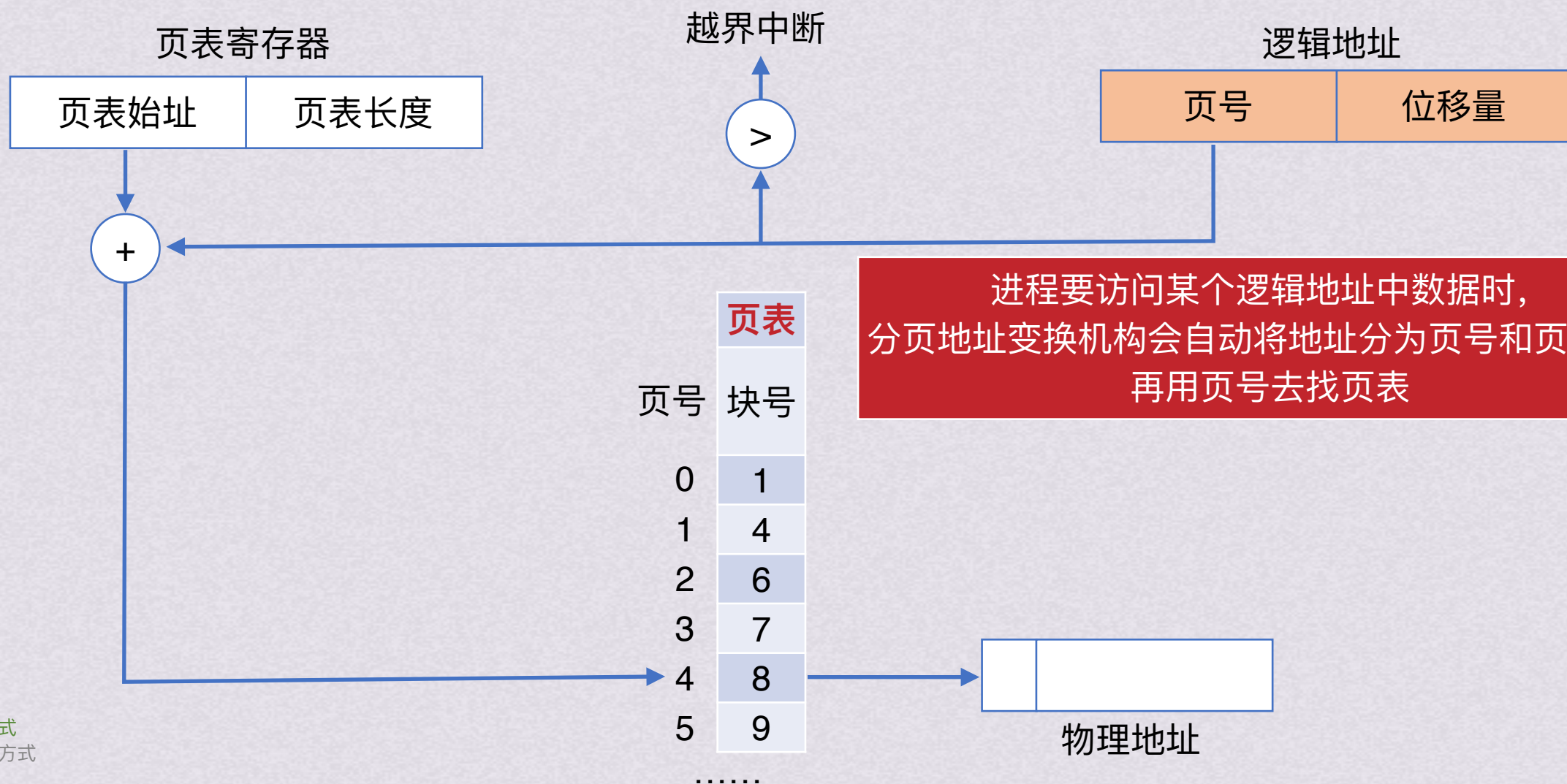
{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式



离散分配存储管理方式

分页存储管理方式

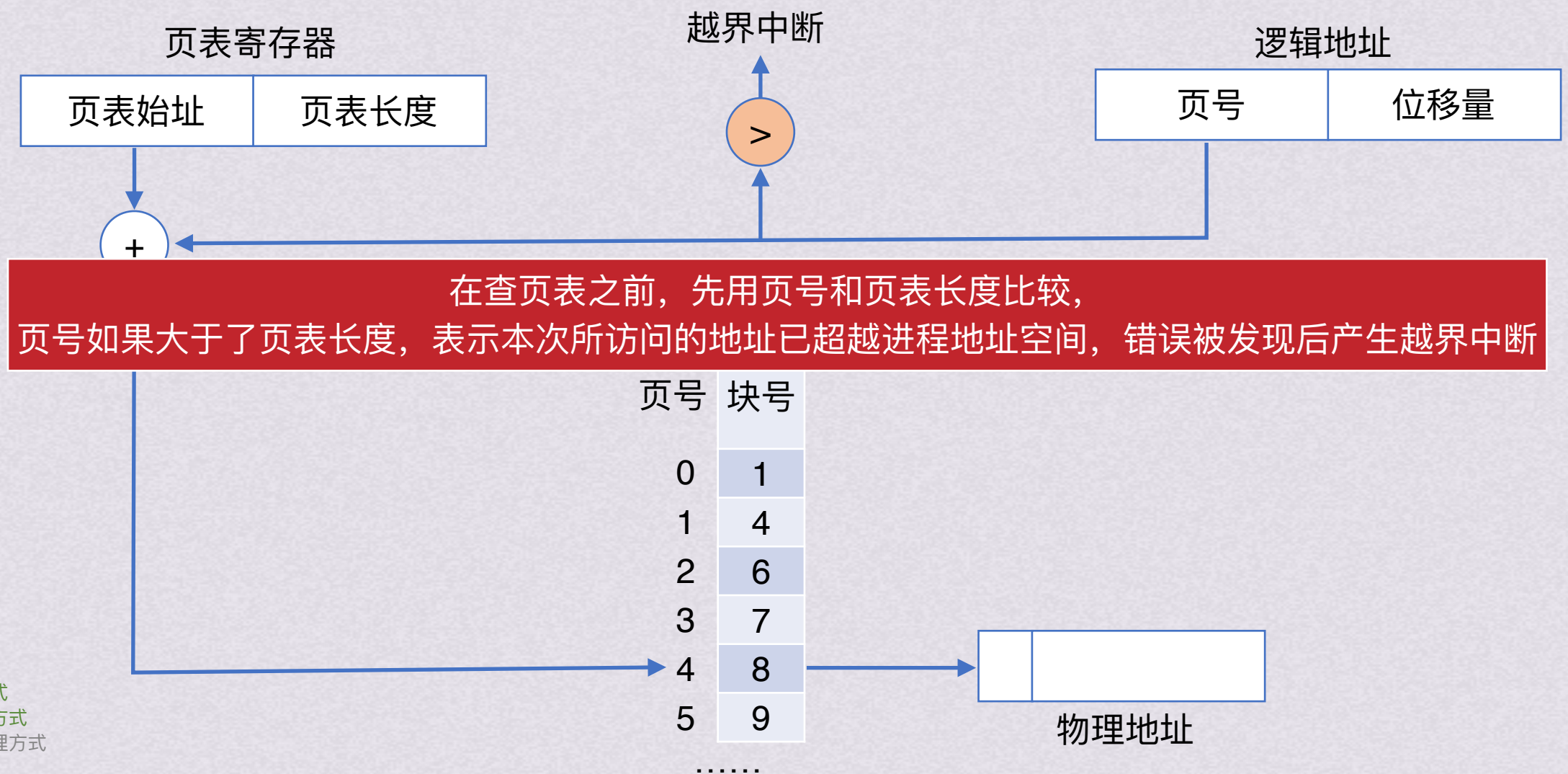
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了

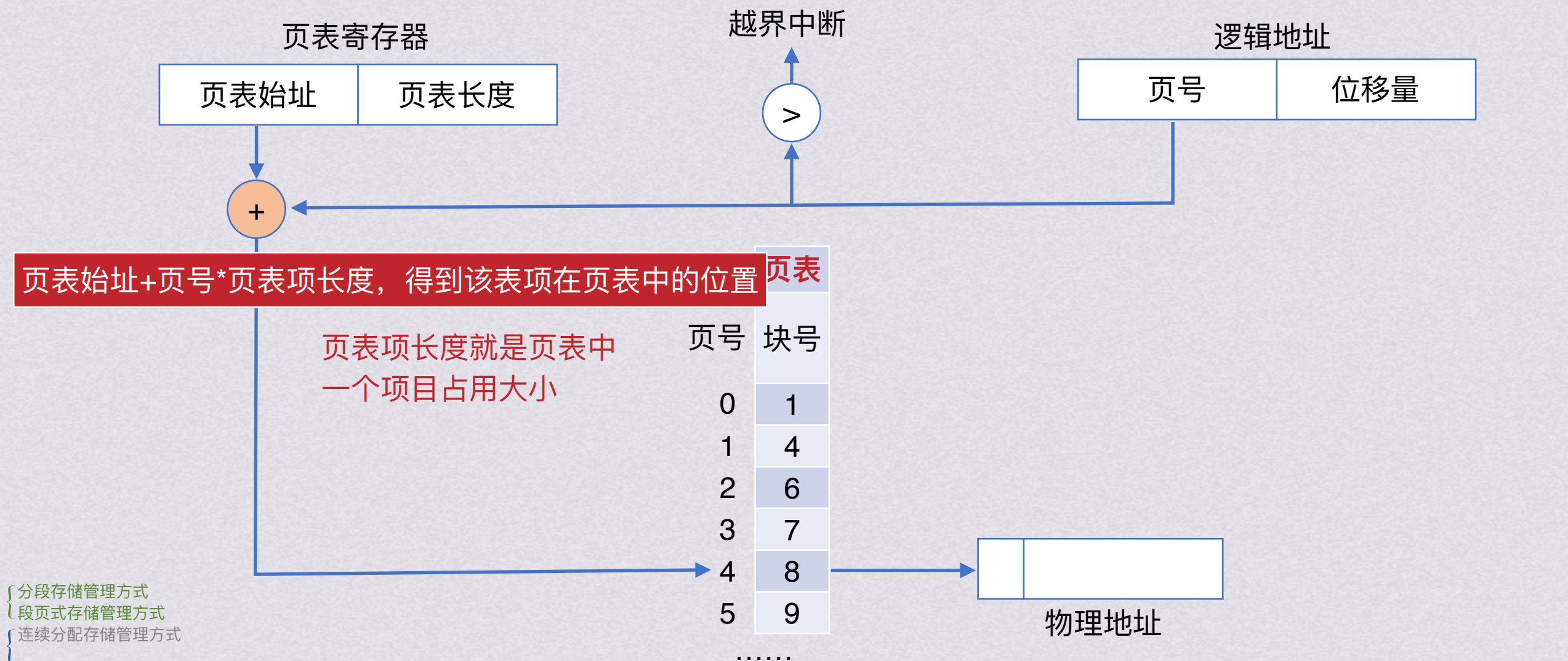


- 分段存储管理方式
- 段页式存储管理方式
- 连续分配存储管理方式

离散分配存储管理方式

分页存储管理方式

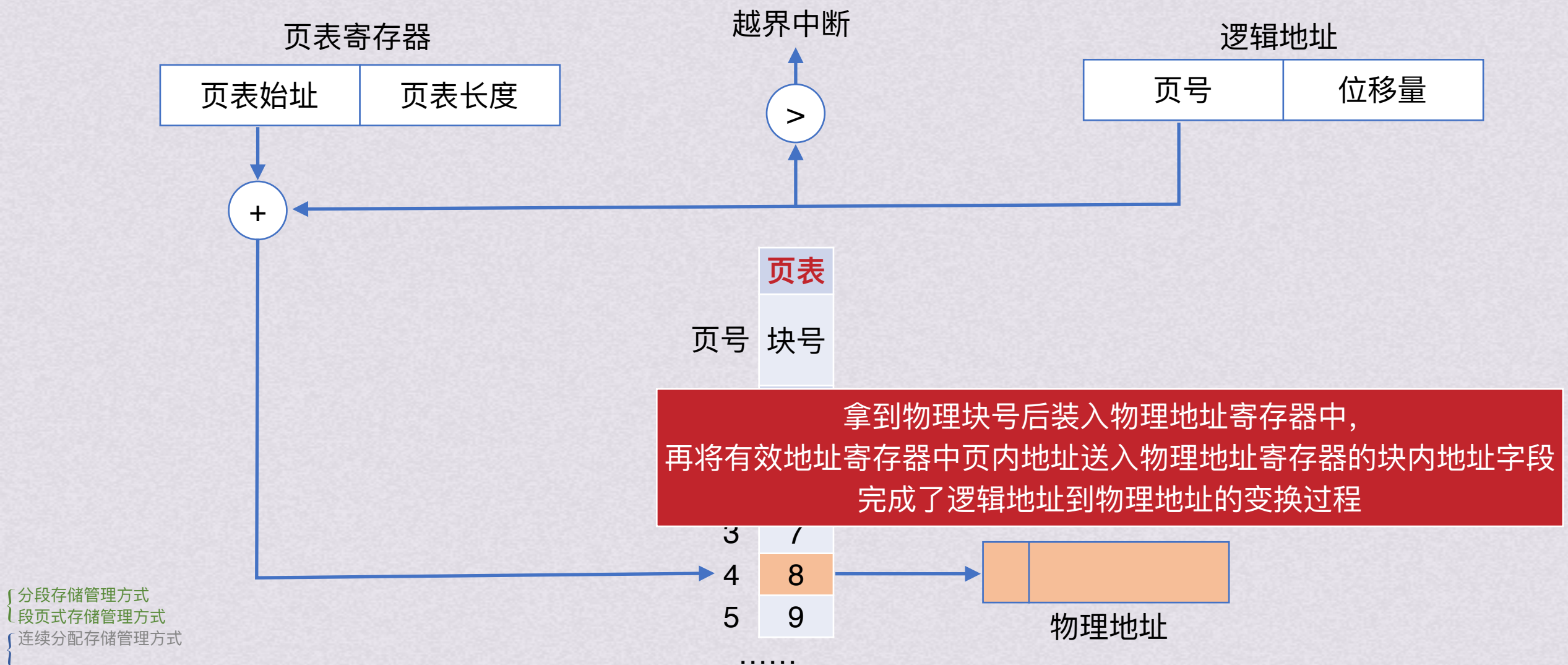
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了

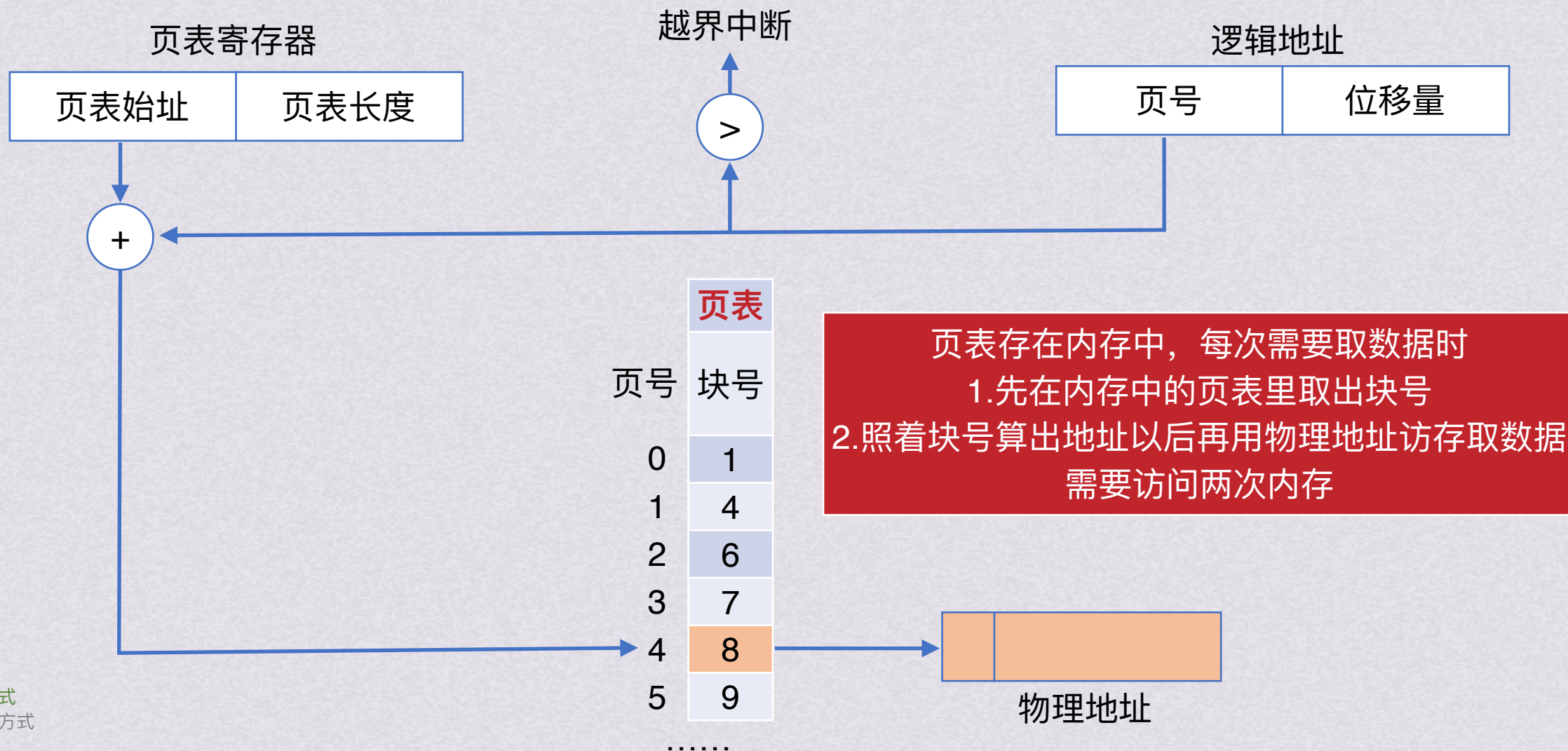




离散分配存储管理方式

分页存储管理方式

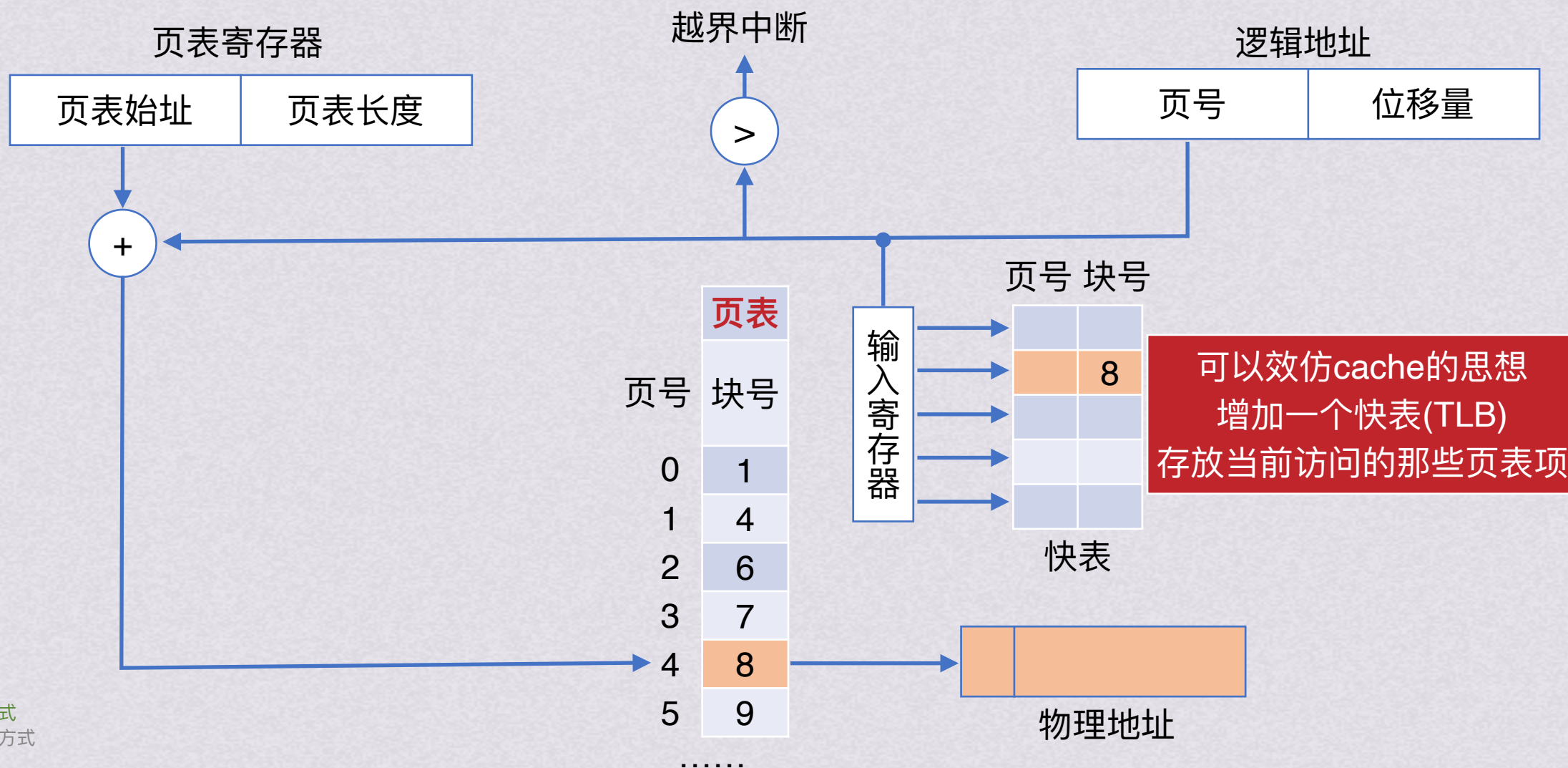
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



离散分配存储管理方式

分页存储管理方式

- 分段存储管理方式
- 段页式存储管理方式
- 连续分配存储管理方式

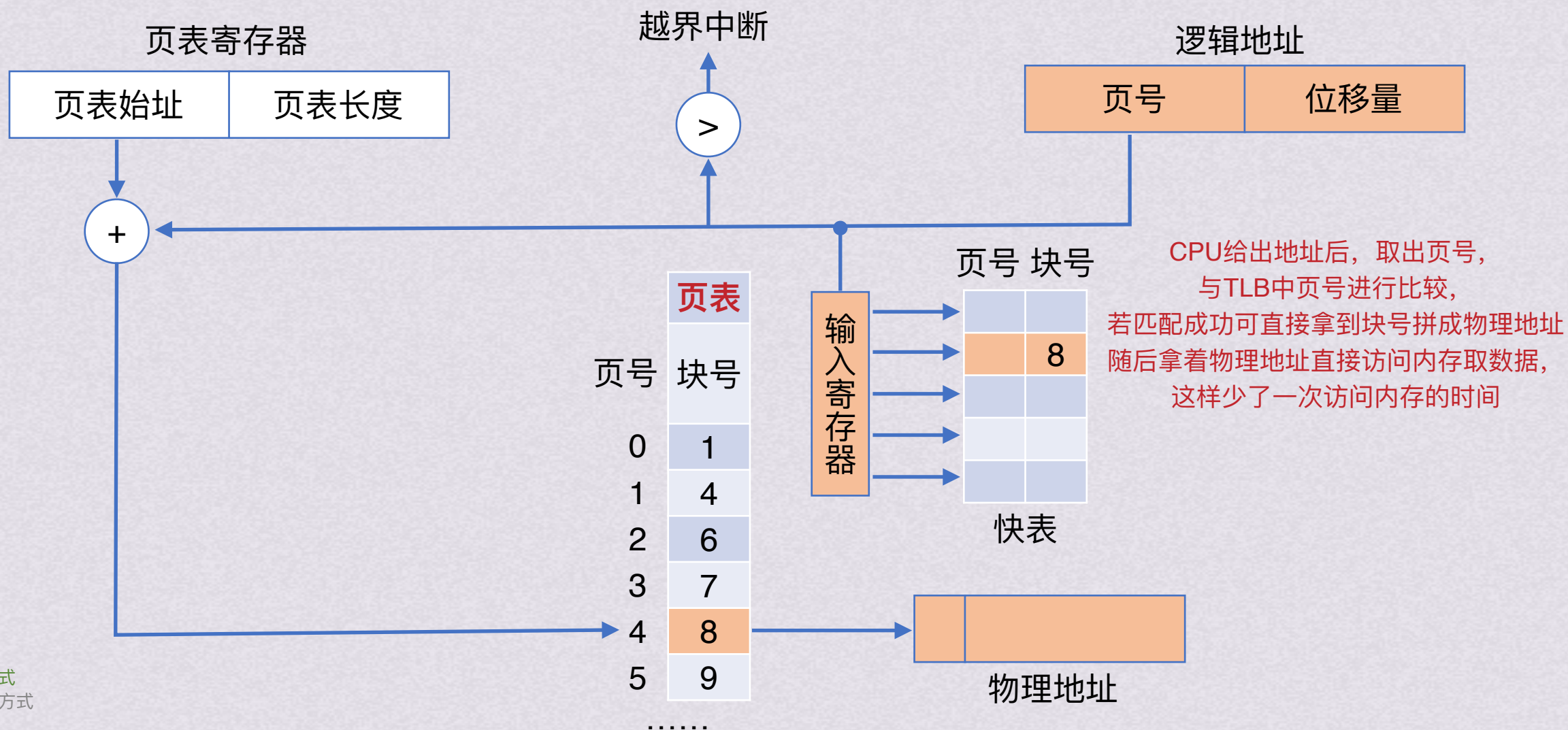




离散分配存储管理方式

分页存储管理方式

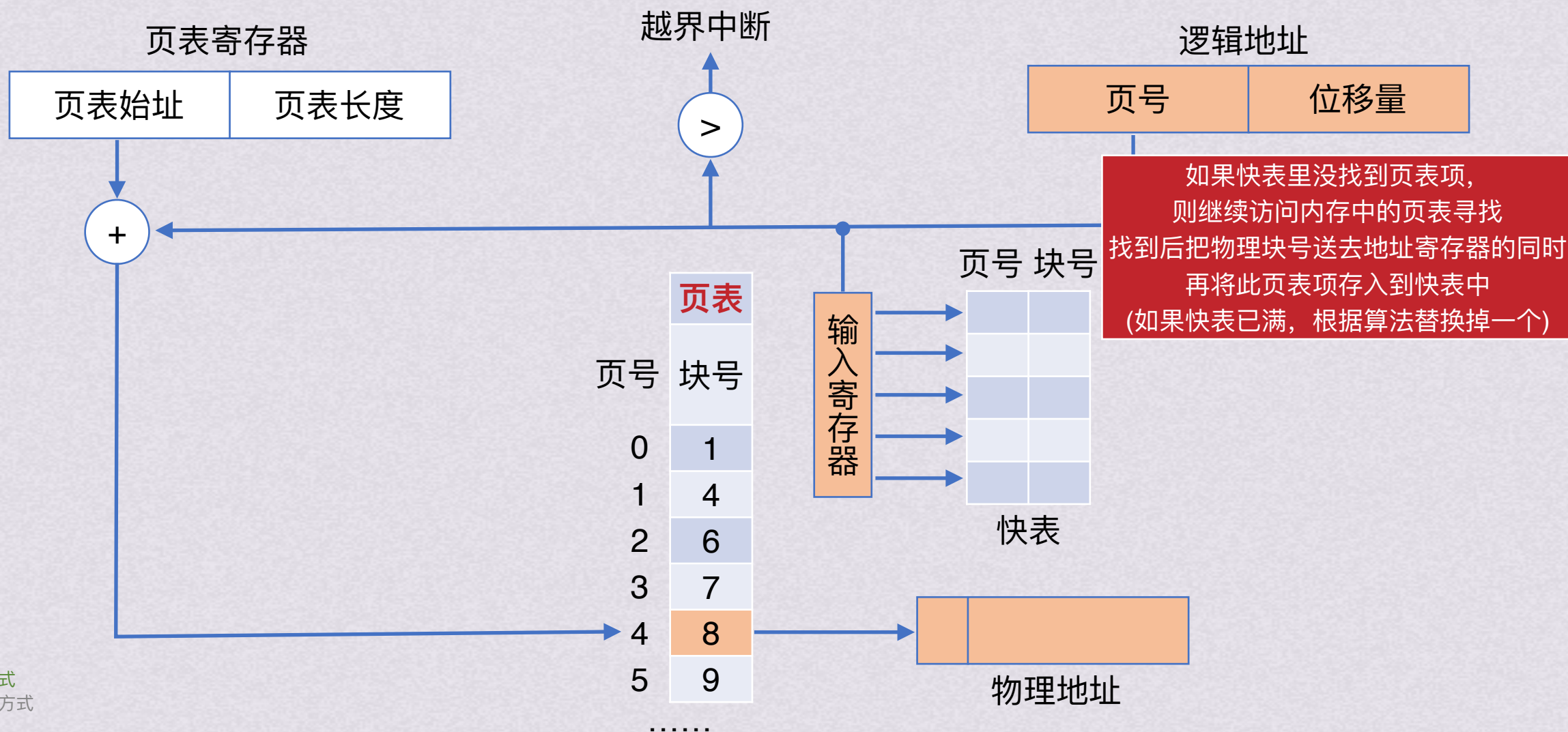
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



离散分配存储管理方式

分页存储管理方式

- 分段存储管理方式
- 段页式存储管理方式
- 连续分配存储管理方式

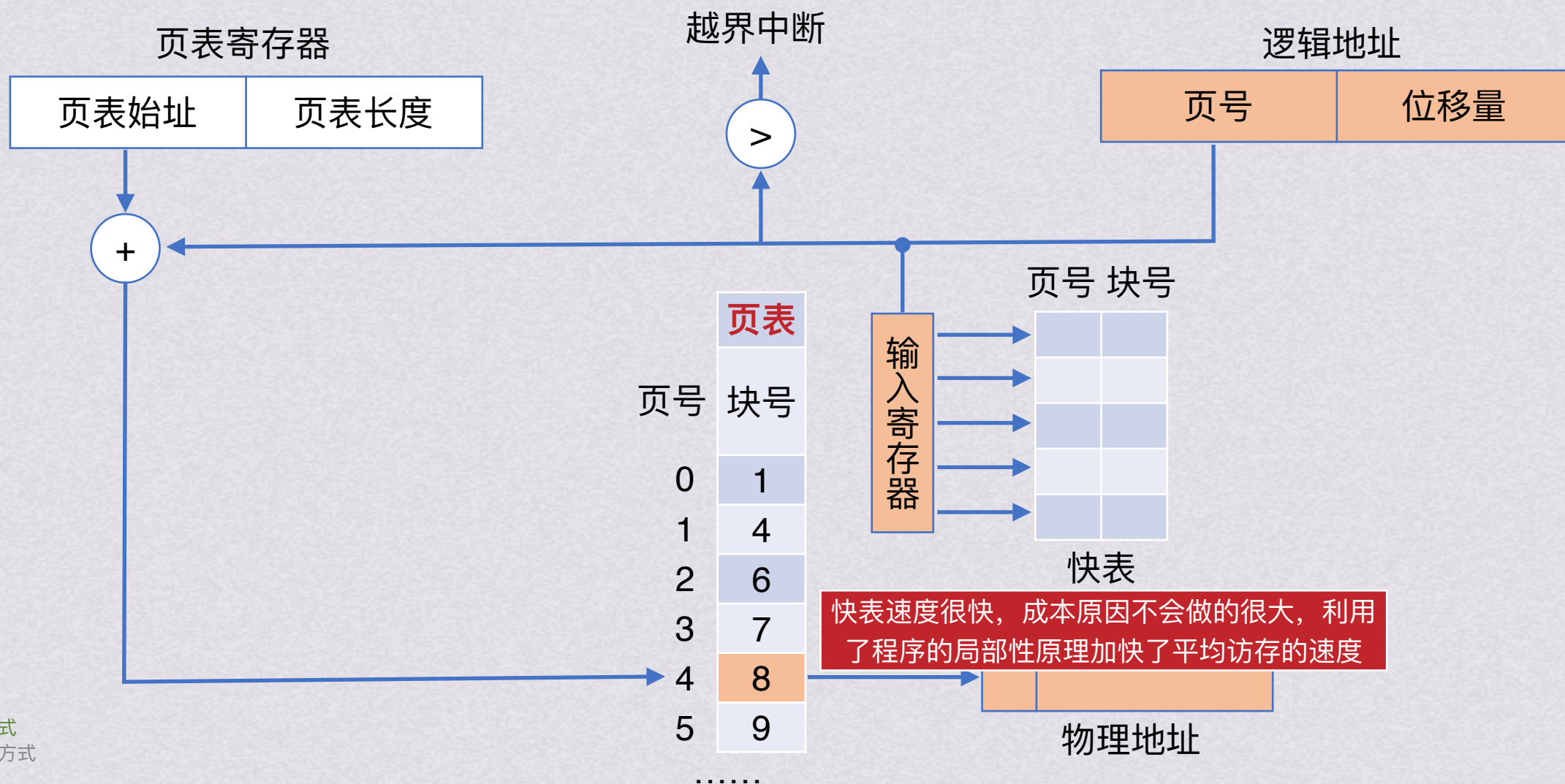




离散分配存储管理方式

分页存储管理方式

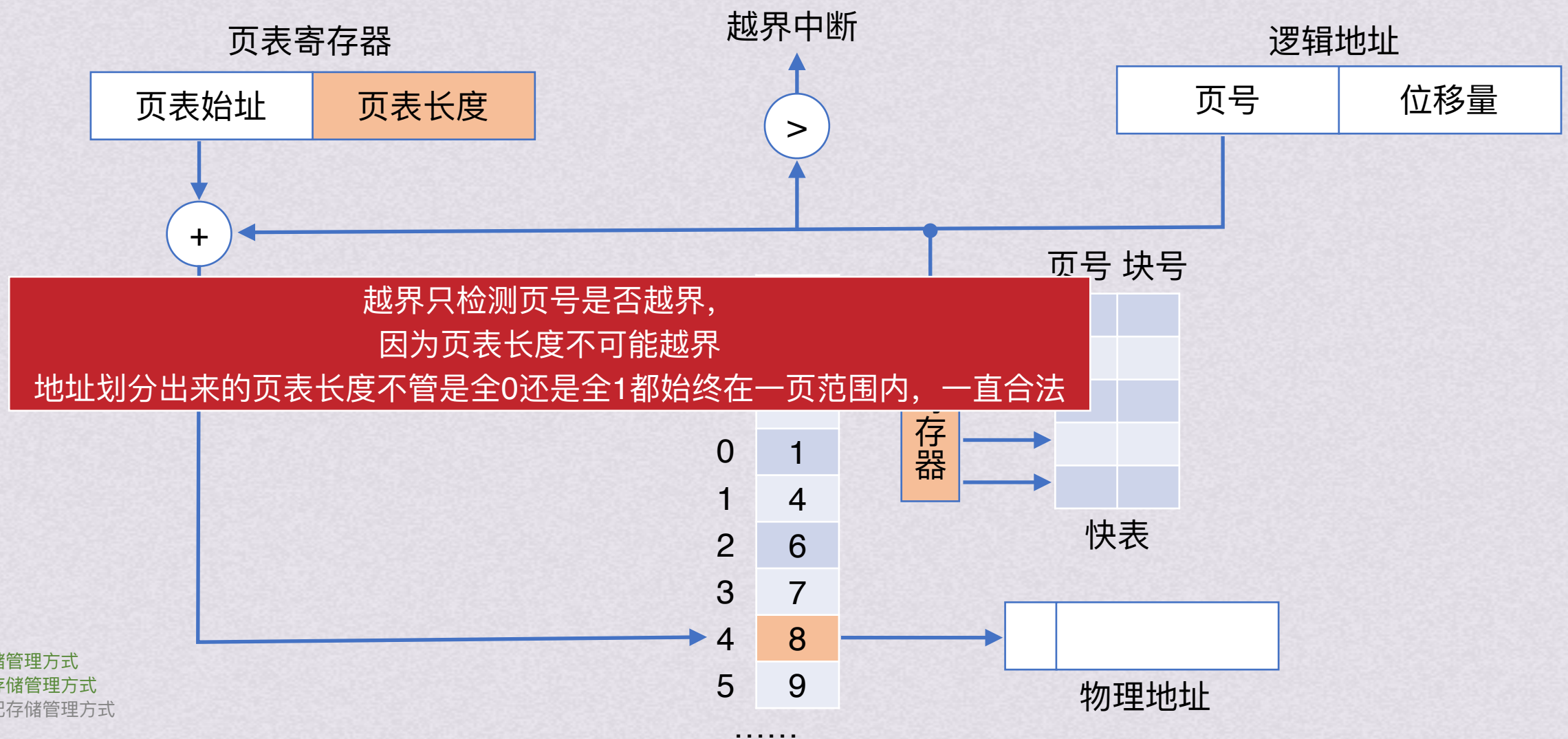
连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



离散分配存储管理方式

分页存储管理方式

连续分配方式会形成许多碎片，虽然可以通过紧凑将碎片拼成大的可用空间，但是会为之付出很大开销所以不如直接把进程切碎了分散地装进许多不相邻的区域中，就可以充分利用内存空间，无须“紧凑”了



- 分段存储管理方式
- 段页式存储管理方式
- 连续分配存储管理方式



分页存储管理方式

页&页表项

页表

页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
...	...
10231	2077
10232	9877



分页存储管理方式

页&页表项

1. 页表是由很多个页表项组成的，右边页表中每一行都是一个页表项

页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
...	...
10231	2077
10232	9877



分页存储管理方式

页&页表项

1. 页表是由很多个页表项组成的，右边页表中每一行都是一个页表项

页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
...	...
10231	2077
10232	9877

页表项0



分页存储管理方式

页&页表项

1. 页表是由很多个页表项组成的，右边页表中每一行都是一个页表项

页表	
页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
...	...
10231	2077
10232	9877

页表项0

页表项1



分页存储管理方式

页&页表项

1. 页表是由很多个页表项组成的，右边页表中每一行都是一个页表项

页表		
页号	块号	
0	1	页表项0
1	4	页表项1
2	6	页表项2
3	7	
4	8	
5	9	
...	...	
10231	2077	
10232	9877	



分页存储管理方式

页&页表项

1. 页表是由很多个页表项组成的，右边页表中每一行都是一个页表项
2. 每个页表项中记录了一个页面被存在了哪一个物理块里

页表		
页号	块号	
0	1	页表项0
1	4	页表项1
2	6	页表项2
3	7	页表项3
4	8	页表项4
5	9	页表项5
...	...	
10231	2077	页表项10231
10232	9877	页表项10232



分页存储管理方式

页&页表项

1.页表是由很多个页表项组成的，右边页表中每一行都是一个页表项

2.每个页表项中记录了一个页面被存在了哪一个物理块里





分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

假设有32位逻辑地址, 页面大小为4KB(2^{12}B), 则有20bit用来表示页号, 即有 2^{20} 页, 每个页表项占用一个字节, 则每个进程仅页表就要占用1MB的连续的内存空间

页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
...	...
10231	2077
10232	9877

所以从两个角度来解决这个问题:

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置

2. 只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入



分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

假设有32位逻辑地址, 页面大小为4KB(2^{12}B), 则有20bit用来表示页号, 即有 2^{20} 页,

每个页表项占用一个字节, 则每个进程仅页表就要占用1MB的连续的内存空间

所以从两个角度来解决这个问题: 1.将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置
2.只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入

页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
...	...
10231	2077
10232	9877



分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

假设有32位逻辑地址, 页面大小为4KB(2^{12}B), 则有20bit用来表示页号, 即有 2^{20} 页,

每个页表项占用一个字节, 则每个进程仅页表就要占用1MB的连续的内存空间

所以从两个角度来解决这个问题: 1.将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置
2.只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入

页表编号	块号
0	1011
1	1078
2	...
3	...
...	...
n	1742

页号	块号
0	114
1	115
2	...
...	...
1023	...

页号	块号
0	1
1	4
2	6
...	...
1023	...

⋮

新建一张表来记录这些被切后的页表所在位置

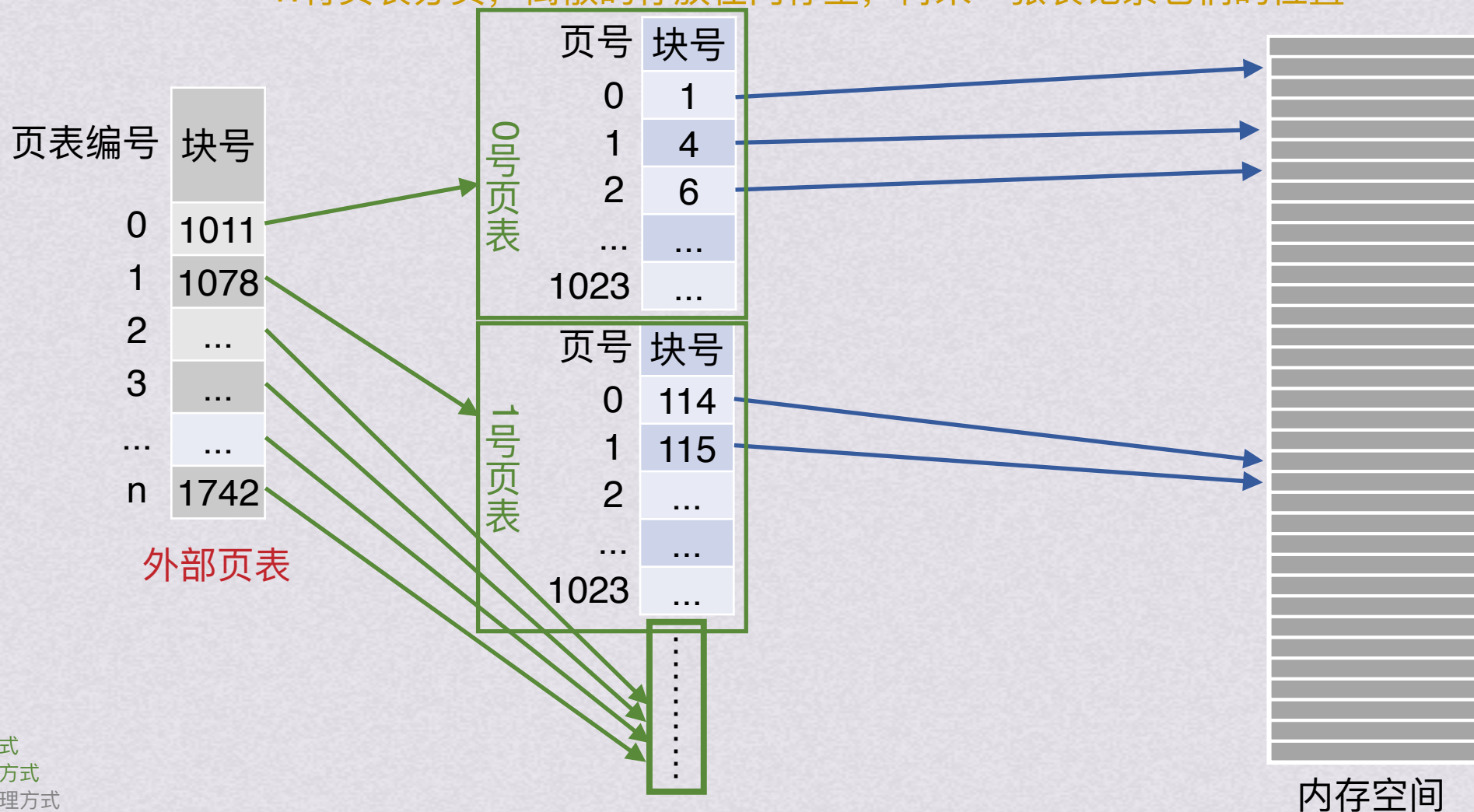


分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置



{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

2. 只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入

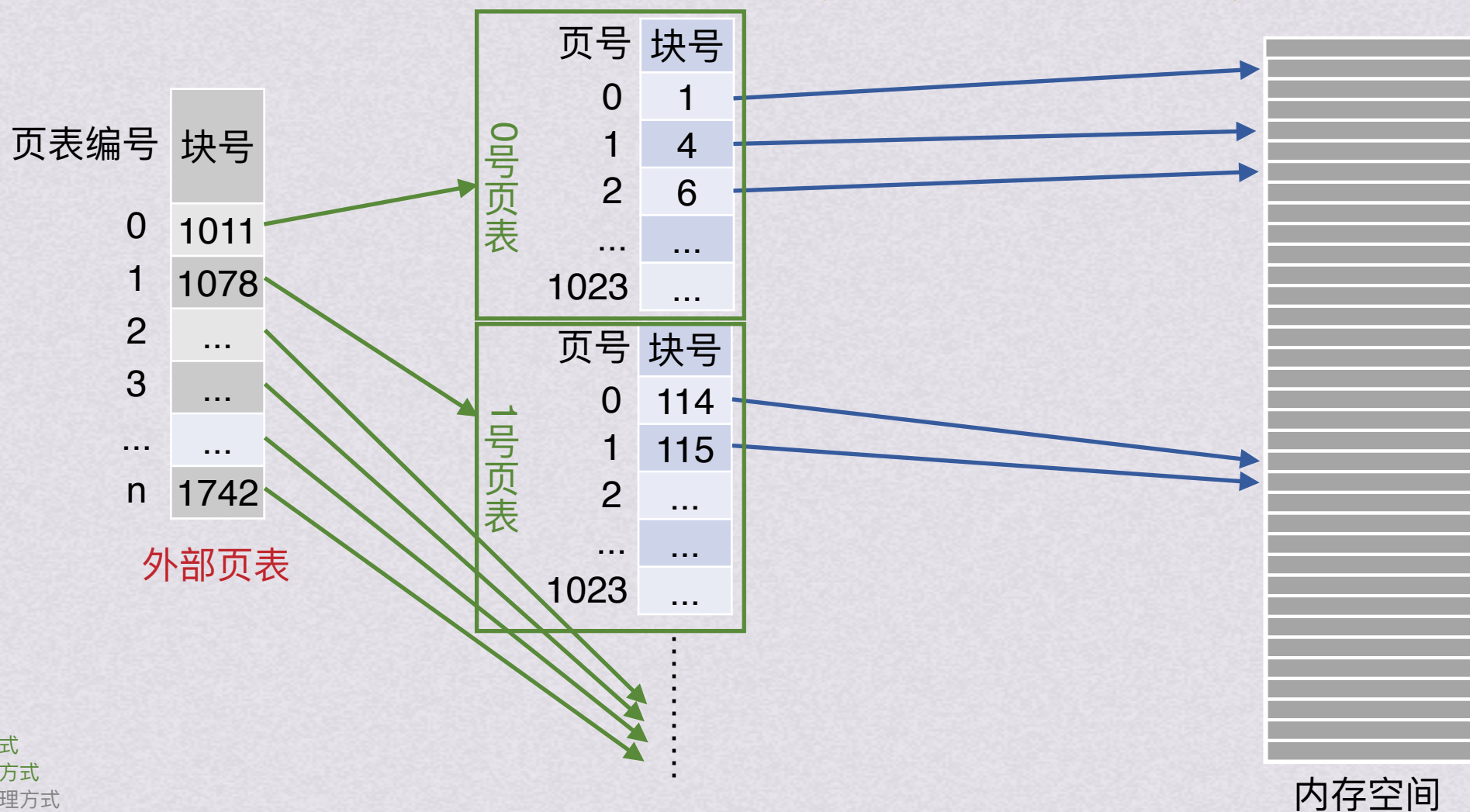


分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

2. 只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入



{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置



分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

需要新增一个有效位来说明目标页表是否在内存中

页表编号	是否有效	块号
0	1	1011
1	1	1078
2	0	-
3	...	...
...	0	-
n	1	1742

外部页表

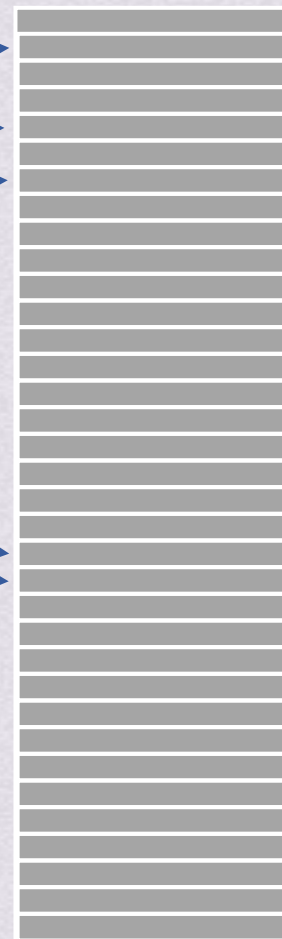
通常最高级的页表都只有一张

0号页表

0	1
1	4
2	6
...	...
1023	...

1号页表

页号	块号
0	114
1	115
2	...
...	...
1023	...



内存空间

{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置



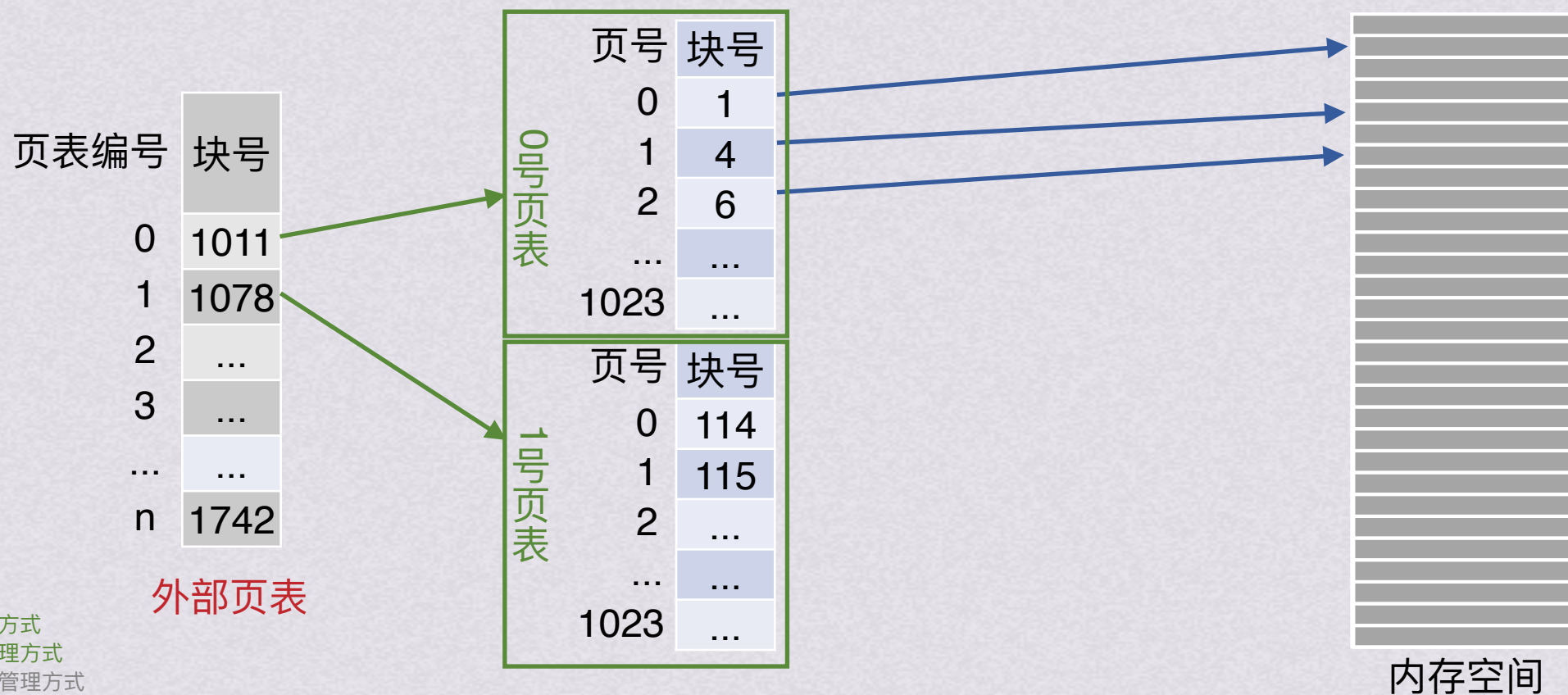
分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

2. 只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入

页目录号	页号	页内偏移
------	----	------



{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置



分页存储管理方式

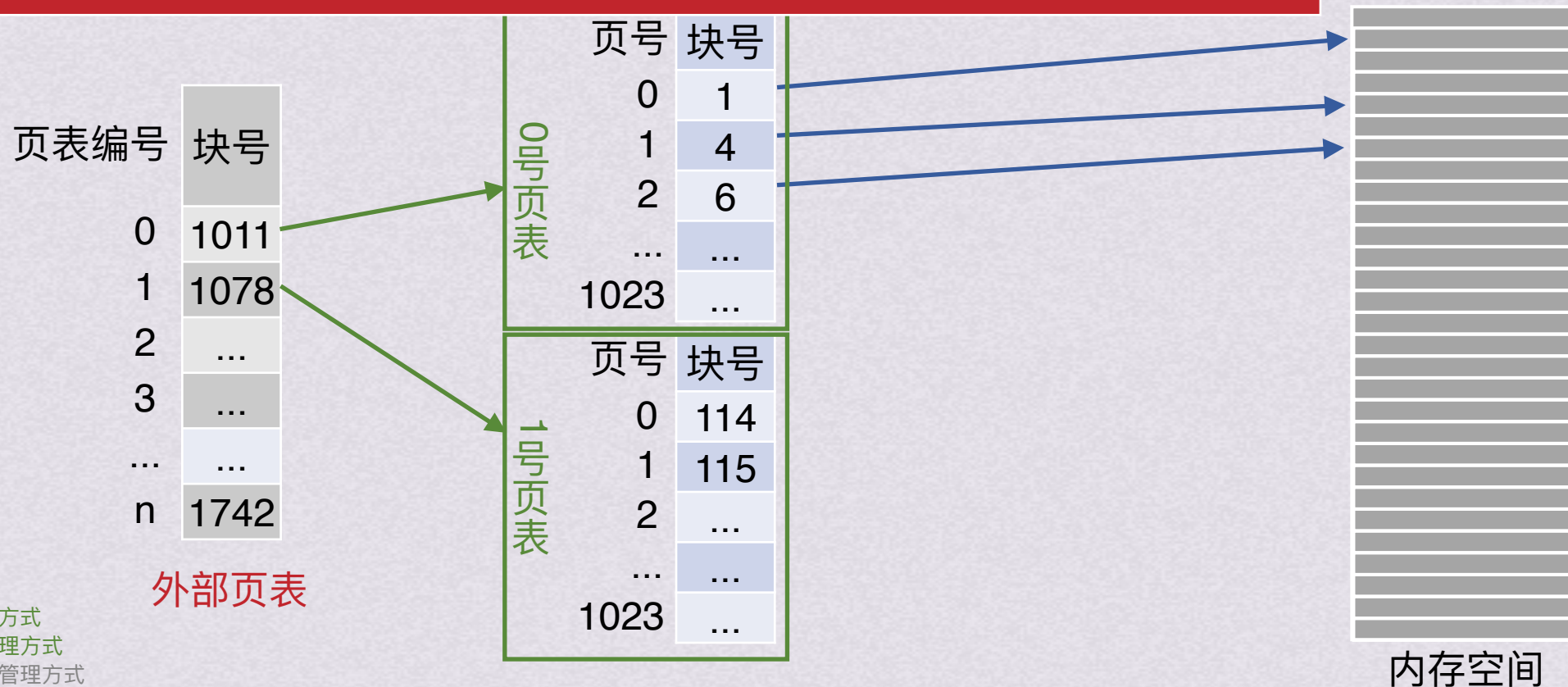
二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

2. 只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入

页目录号	页号	页内偏移
------	----	------

页目录号用来指示在页目录表中的页表编号, 从而拿到二级页表所在物理块地址



{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置

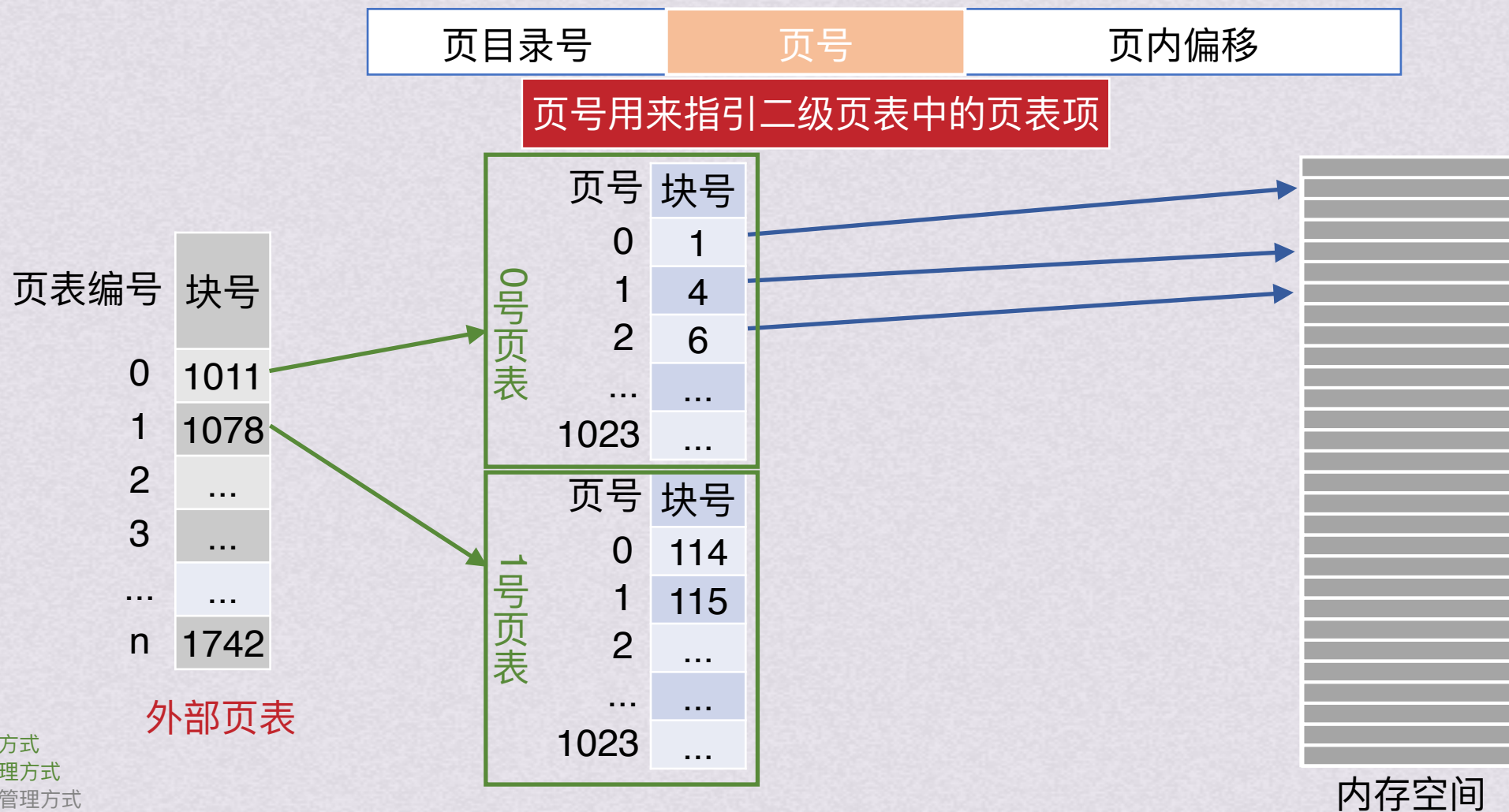


分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

2. 只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入



{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置

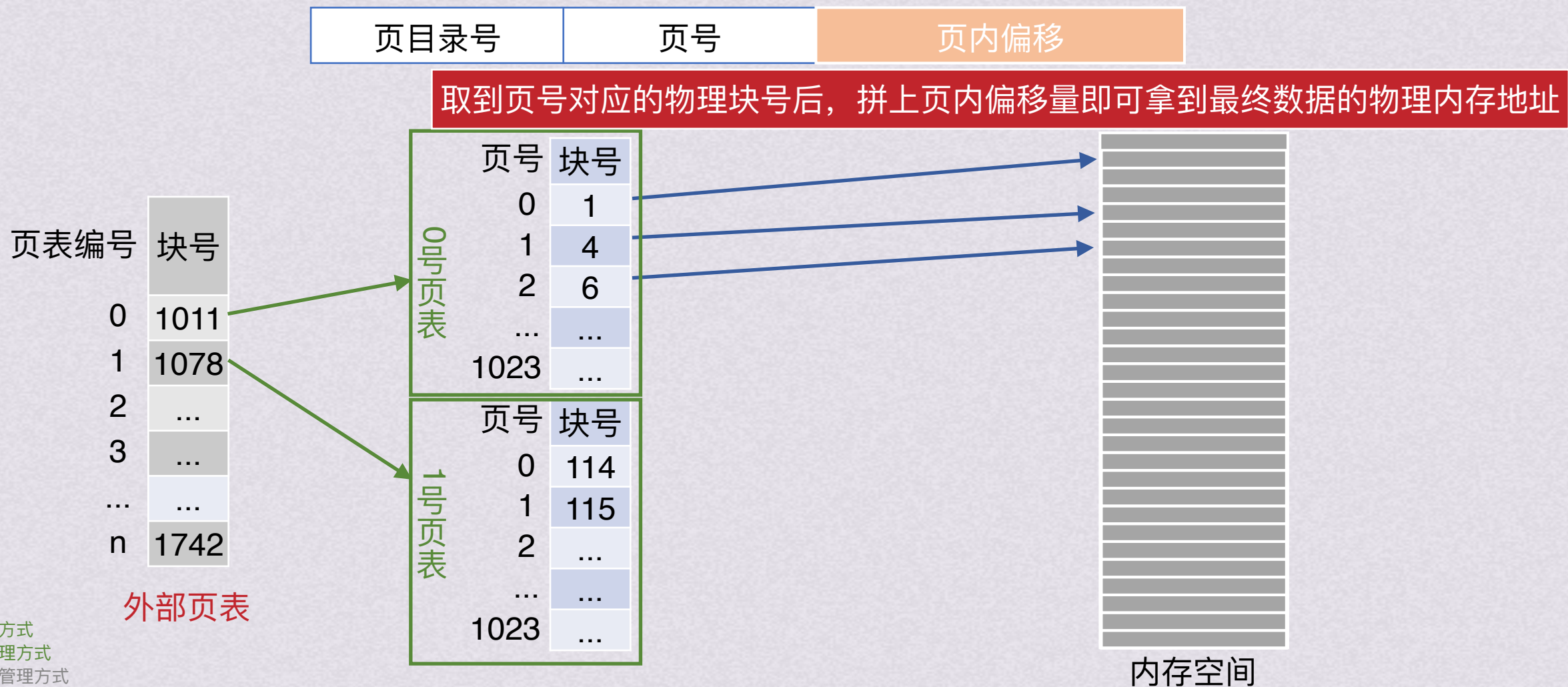


分页存储管理方式

二级和多级页表

现代计算机都支持非常大的逻辑地址空间($2^{32} \sim 2^{64} \text{B}$), 在这样的环境下页表就变得非常大, 也会占用掉巨大内存空间

2. 只把现在要用到的部分页表项调入内存, 其他的页表项放在磁盘, 需要时再调入



{ 分段存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

1. 将页表分页, 离散的存放在内存里, 再来一张表记录它们的位置



二级和多级页表

练一练

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
-----------	---------	-----------

虚拟地址2050 1225H对应的页目录号、页号分别是

A.081H,101H

B.081H,401H

C.201H,101H

D.201H,401H



二级和多级页表

练一练

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
-----------	---------	-----------

虚拟地址2050 1225H对应的页目录号、页号分别是

A.081H,101H



B.081H,401H

C.201H,101H

D.201H,401H

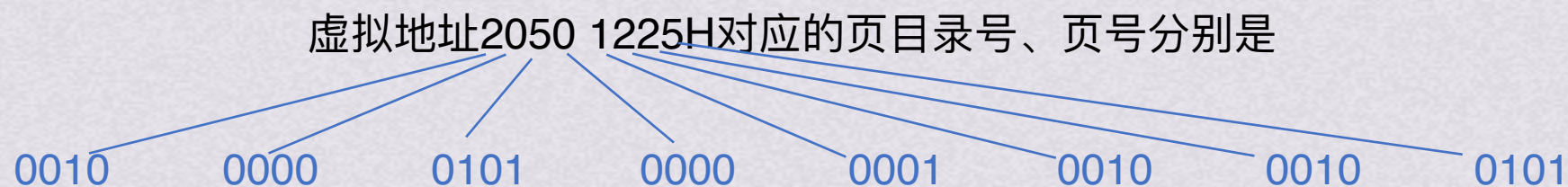


二级和多级页表

练一练

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
-----------	---------	-----------



A.081H,101H

B.081H,401H

C.201H,101H

D.201H,401H



二级和多级页表

练一练

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
-----------	---------	-----------

0010000001

010000 0001

0010 0010 0101

虚拟地址2050 1225H对应的页目录号、页号分别是

A.081H,101H

B.081H,401H

C.201H,101H

D.201H,401H



二级和多级页表

练一练

[2019]某计算机主存按字节编址，采用二级分页存储管理，地址结构如下

页目录号(10位)	页号(10位)	页内偏移(12位)
0010000001	010000 0001	001000100101
081H	101H	225H

虚拟地址2050 1225H对应的页目录号、页号分别是



二级和多级页表

练一练

[2010]某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为 2^{10} B,页表项大小为2B，逻辑地址结构为

页目录号	页号	页内偏移
------	----	------

逻辑地址空间大小为 2^{16} 页，则标识整个逻辑地址空间的页目录表中包含表项的个数至少是

A.64

B.128

C.256

D.512



二级和多级页表

练一练

[2010]某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为 2^{10} B,页表项大小为2B，逻辑地址结构为

页目录号	页号	页内偏移
------	----	------

逻辑地址空间大小为 2^{16} 页，则标识整个逻辑地址空间的页目录表中包含表项的个数至少是

A.64

B.128

C.256

D.512





二级和多级页表

练一练

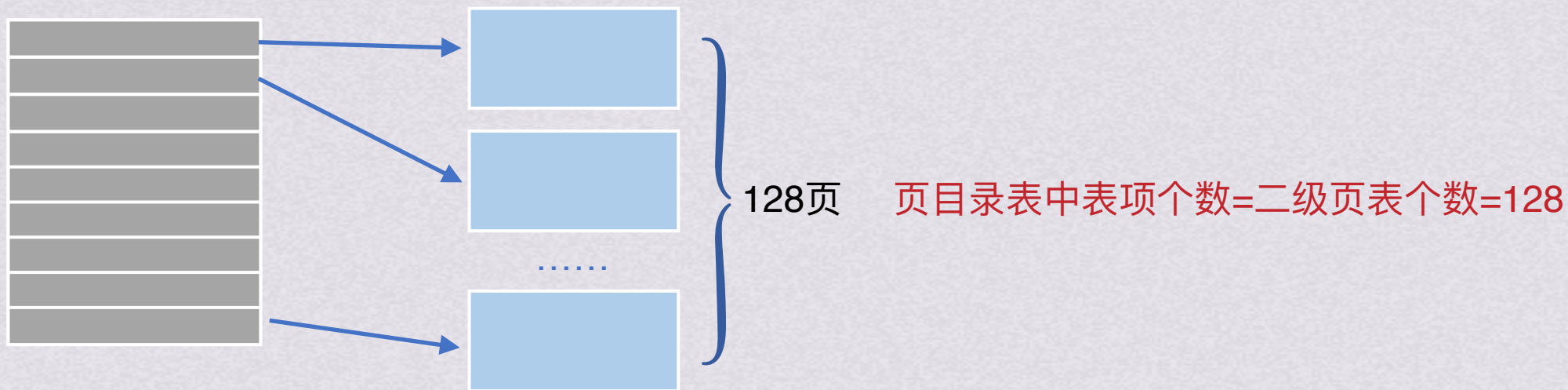
[2010]某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为 $2^{10}B$ ，页表项大小为 $2B$ ，逻辑地址结构为

页目录号	页号	页内偏移
------	----	------

逻辑地址空间大小为 2^{16} 页，则标识整个逻辑地址空间的页目录表中包含表项的个数至少是

逻辑地址空间大小为 2^{16} 页 $\rightarrow$ 页表有 2^{16} 个项

给页表分页，能分 $\frac{2^{16} \times 2B}{2^{10}B} = 2^7 = 128$ 页





离散分配存储管理方式

分页存储管理方式



离散分配存储管理方式

分段存储管理方式



离散分配存储管理方式

分段存储管理方式

之前介绍的分页机制通过硬件机制实现，对用户完全透明。分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段



用户程序

每段都从0开始编址，并且会在内存中被分配一段连续的地址空间
(段内要求连续，段间不要求连续，进程的地址空间是二维的)



离散分配存储管理方式

分段存储管理方式

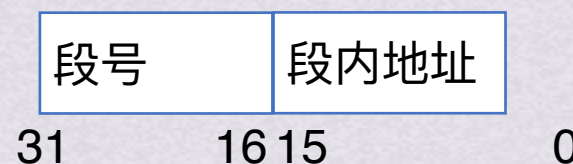
之前介绍的分页机制通过硬件机制实现，对用户完全透明。分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段



用户程序

分段地址的地址结构



该地址允许一个作业最多有 2^{16} 个段
每个段最长 2^{16} B

{ 分页存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式

每段都从0开始编址，并且会在内存中被分配一段连续的地址空间
(段内要求连续，段间不要求连续，进程的地址空间是二维的)



离散分配存储管理方式

分段存储管理方式

之前介绍的分页机制通过硬件机制实现，对用户完全透明。分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段



用户程序



内存

{ 分页存储管理方式
{ 段页式存储管理方式
{ 连续分配存储管理方式



离散分配存储管理方式

分段存储管理方式

之前介绍的分页机制通过硬件机制实现，对用户完全透明。分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段

同样，需要为每个进程建立一张段映射表，简称段表

段号	段长	基址
0	A	40K
1	B	60K
2	C	120K
3	D	50K
4	E	299K

段表

用户程序



内存

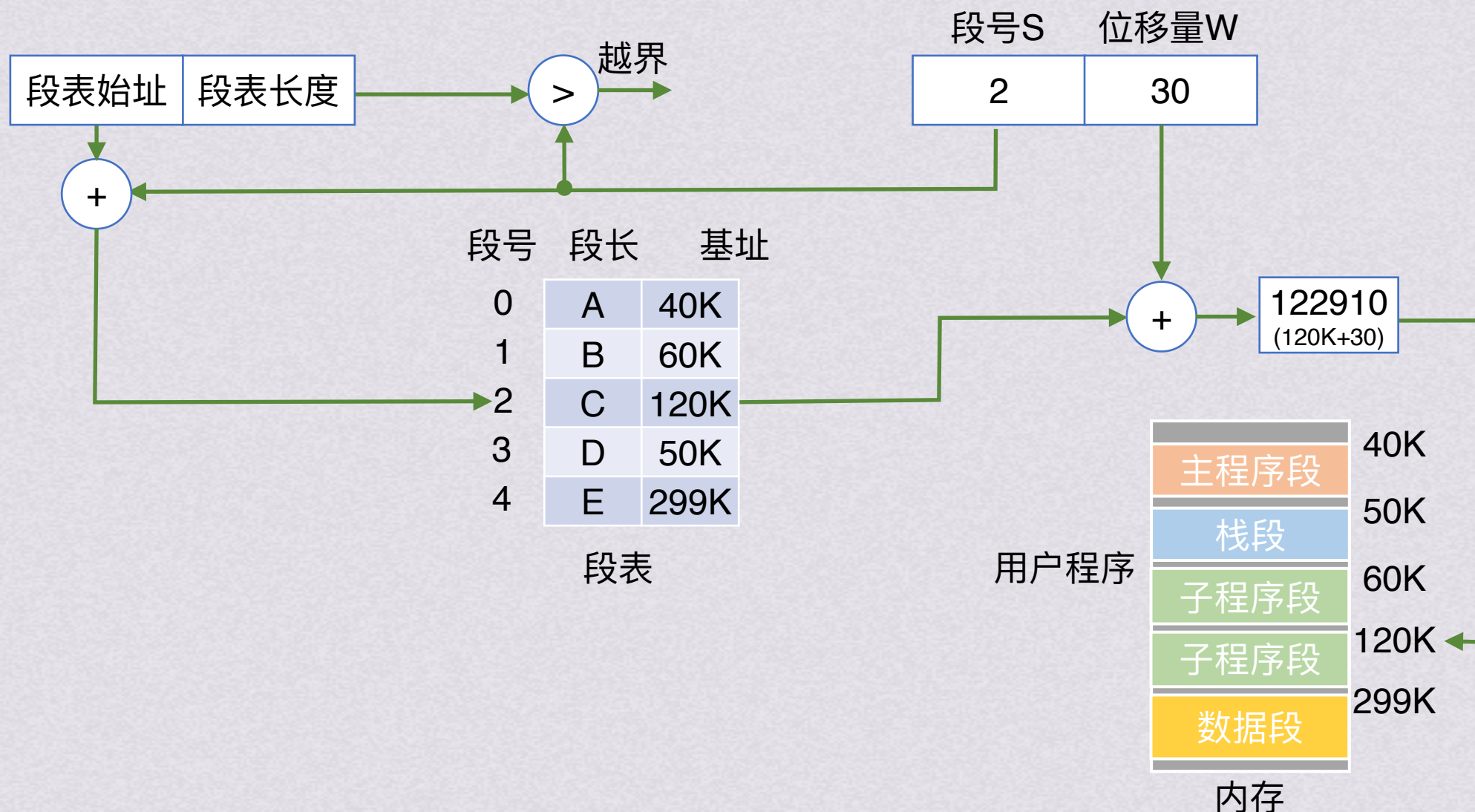


离散分配存储管理方式

分段存储管理方式

之前介绍的分页机制通过硬件机制实现，对用户完全透明。分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段



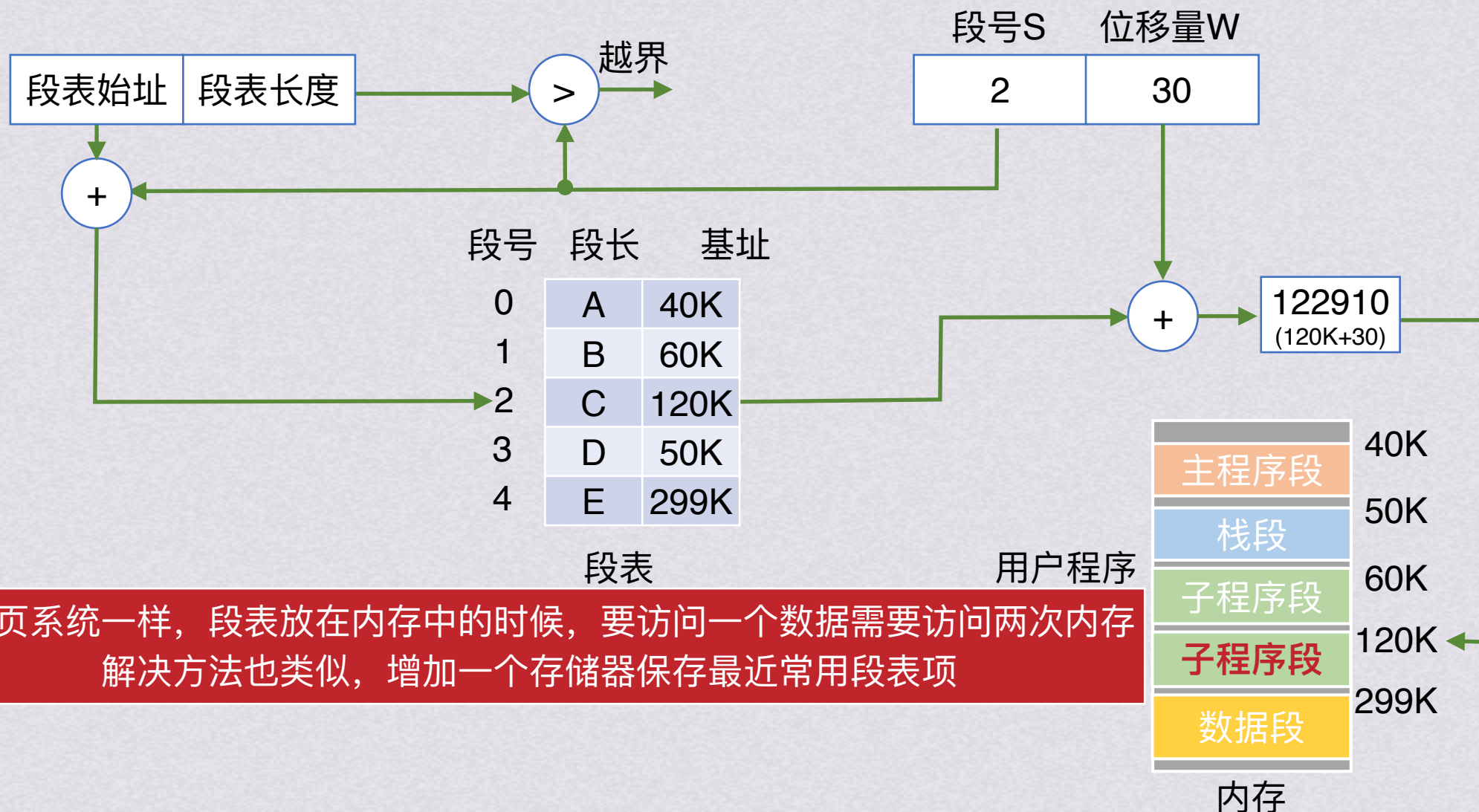


离散分配存储管理方式

分段存储管理方式

之前介绍的分页机制通过硬件机制实现，对用户完全透明。分段管理方式则考虑了用户和程序员

分段系统将用户进程的逻辑地址空间划分为大小不等的段



和分页系统一样，段表放在内存中的时候，要访问一个数据需要访问两次内存
解决方法也类似，增加一个存储器保存最近常用段表项



离散分配存储管理方式

分页存储管理方式

页是信息的物理单位

分页存储是为了离散的放在内存，
提高内存利用率

分页是系统行为，用户不可见

页的大小固定，系统决定

分页的用户程序地址空间是一维的
用户直接给逻辑地址，硬件划分地址

分段存储管理方式

段是信息的逻辑单位

分段是为了更好地满足用户需要

对用户不完全透明

段大小不一，取决于程序

用户程序的地址空间是二维的，
需要段名+段内地址

{ 段页式存储管理方式
{ 连续分配存储管理方式



离散分配存储管理方式

分段存储管理方式

- { 分页存储管理方式
- { 段页式存储管理方式
- { 连续分配存储管理方式



离散分配存储管理方式

段页式存储管理方式



离散分配存储管理方式

段页式存储管理方式

分页用页做内存分配基本单位，有效提高了内存利用率

分段用段作为内存分配基本单位，能满足用户多方面需要



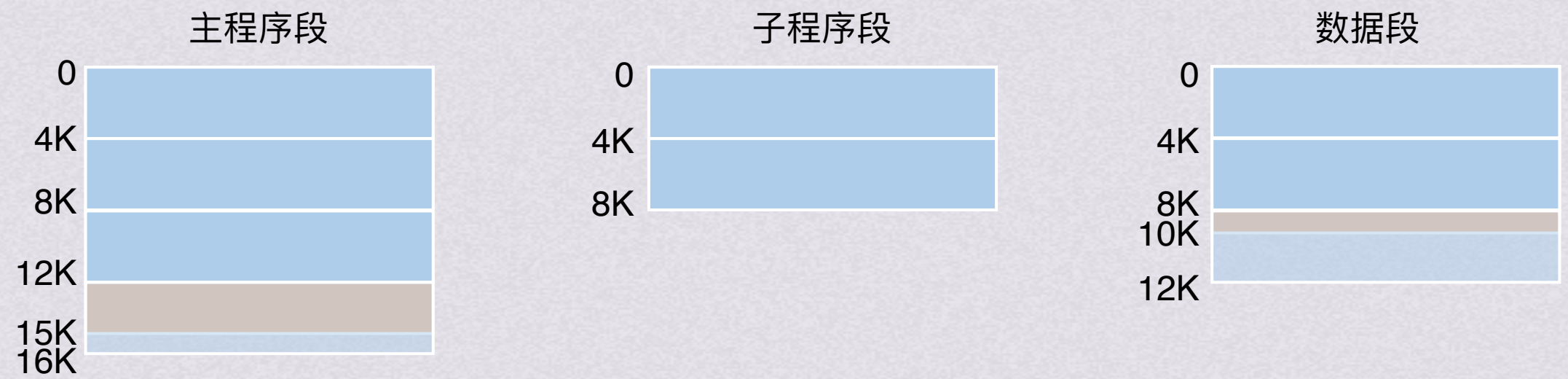
缝合成为全新存储器管理方式：段页式存储管理方式



离散分配存储管理方式

段页式存储管理方式

先把用户程序分成段，再把每个段分成页，并为每个段赋予一个段名



段号S	段内页号P	页内地址W
-----	-------	-------

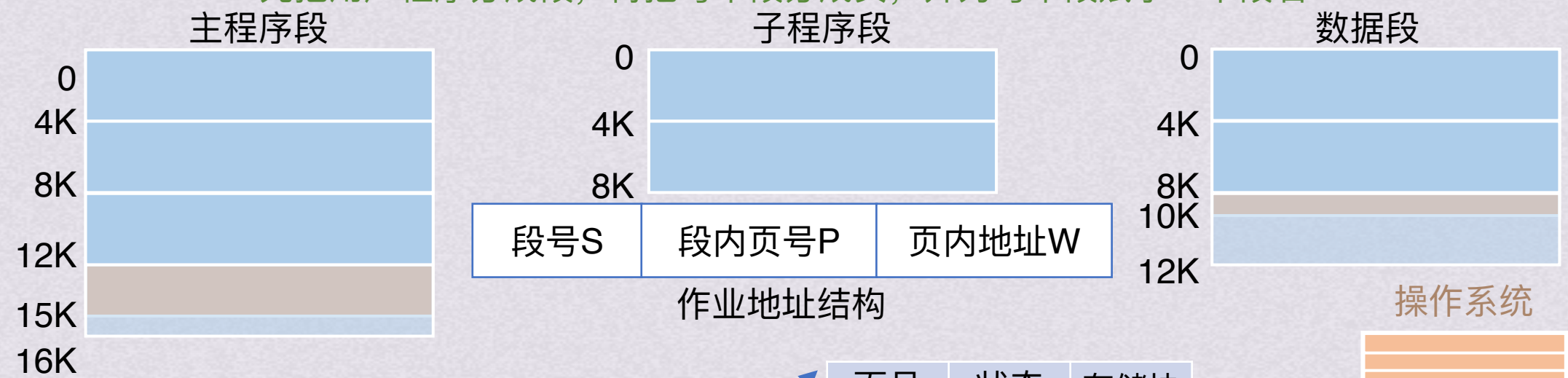
作业地址结构

{ 分页存储管理方式
{ 分段存储管理方式
{ 连续分配存储管理方式

离散分配存储管理方式

段页式存储管理方式

先把用户程序分成段，再把每个段分成页，并为每个段赋予一个段名



段号S	段内页号P	页内地址W
-----	-------	-------

作业地址结构

段表大小	段表始址
------	------

段号	状态	页表大小	页表始址
0	1		
1	1		
2	1		
3	0		
4	1		

页号	状态	存储块
0	1	
1	1	
2	1	
3	0	
4	1	

页号	状态	存储块



内存空间

分页存储管理方式
分段存储管理方式
连续分配存储管理方式

离散分配存储管理方式 段页式存储管理方式

先把用户程序分成段，再把每个段分成页，并为每个段赋予一个段名

每个进程建立一张段表，每个段对应一个段表项，段表项至少包括段号、页表长度、页表始址

每个段有一张页表，每个页表项至少包括页号和块号

还需要一个段表寄存器，指出段表始址和段表长度

段表寄存器和页表寄存器的作用：
1.在段表(页表)中寻址
2.判断是否越界



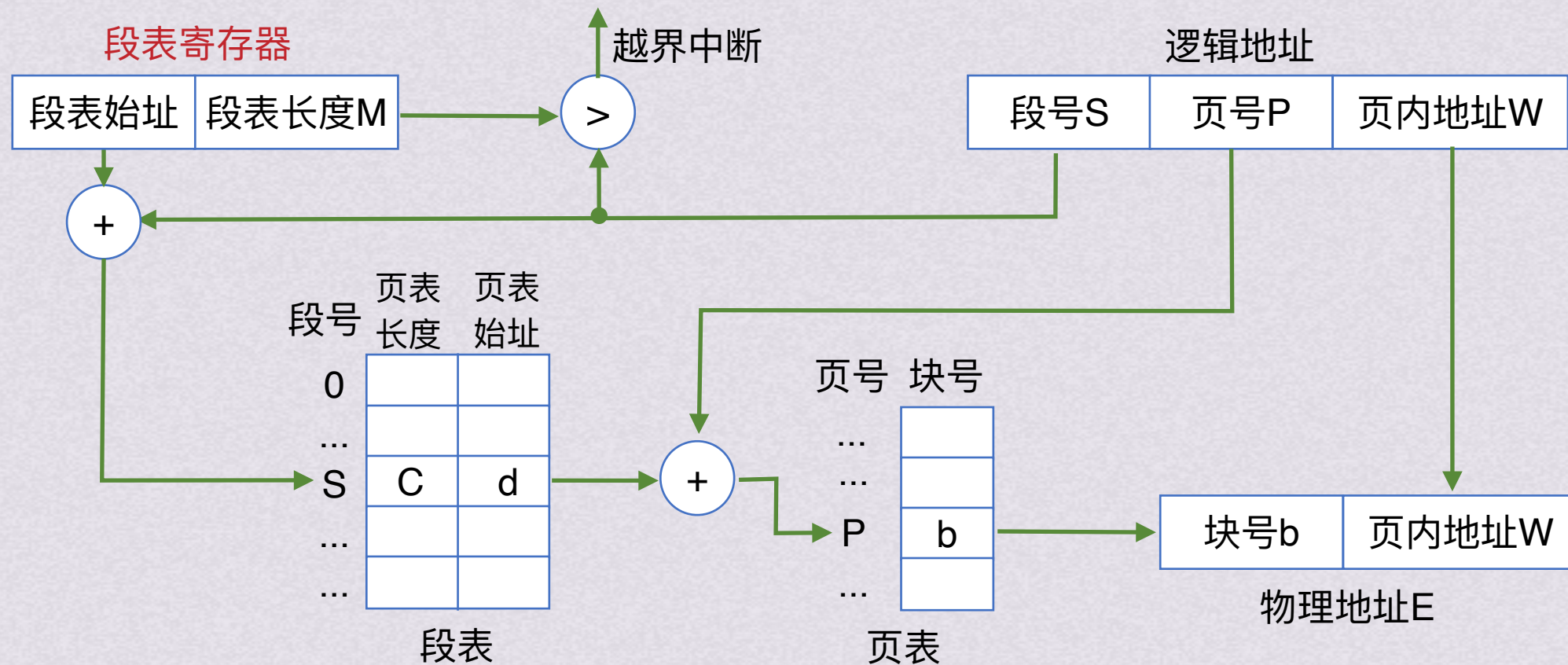
{ 分页存储管理方式
分段存储管理方式
连续分配存储管理方式



离散分配存储管理方式

段页式存储管理方式

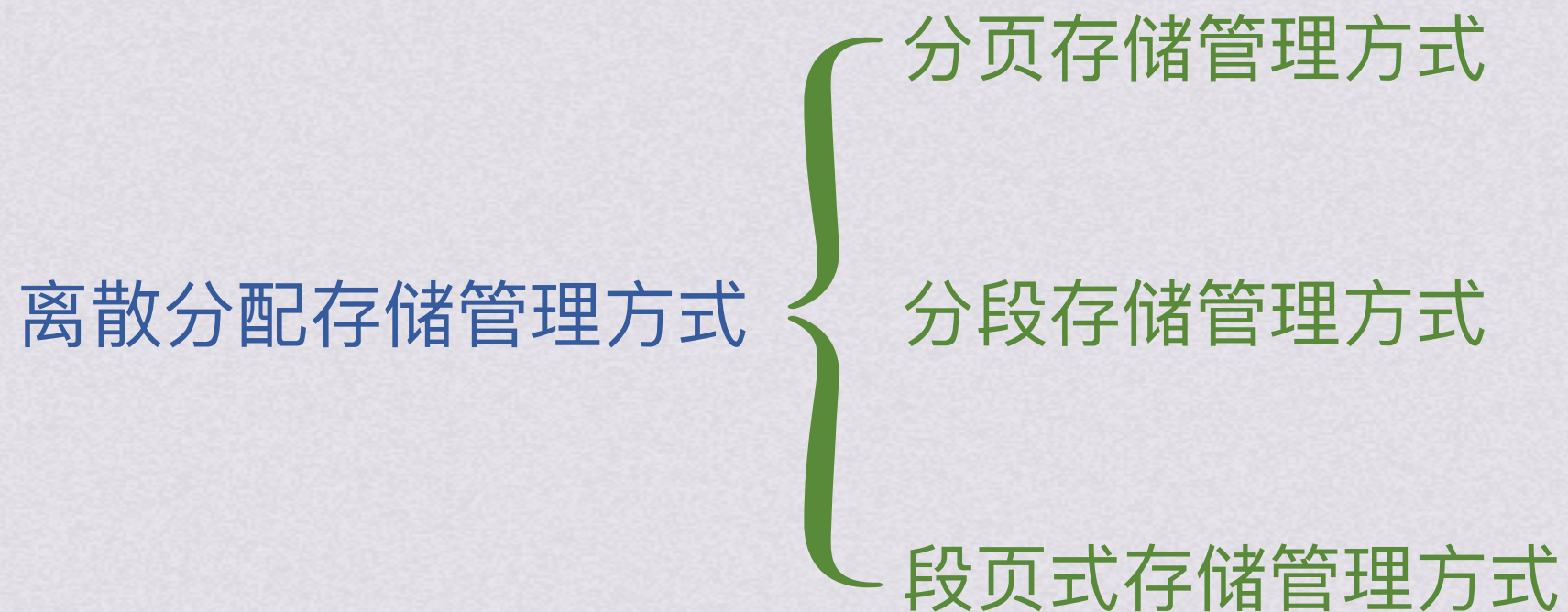
先把用户程序分成段，再把每个段分成页，并为每个段赋予一个段名





离散分配存储管理方式

段页式存储管理方式





离散分配存储管理方式

